

Chapter 1 — Introduction to Databases

Data and Information, Databases, DBMS, Database Applications and View of Data

Unit 1 — what these notes cover

1. Introduction to Databases — data vs. information, database, DBMS, applications, OLTP/OLAP, view of data.
2. Purpose of Database Systems — file-processing systems, their seven drawbacks, characteristics and advantages of the database approach.
3. Database Design — three-schema architecture, data independence, schema vs. instance, users, DBMS modules, architectures.
4. Entity-Relationship Model — entities, attribute types, relationships, weak entities, cardinality, participation, min-max notation.
5. Converting an E-R Diagram to a Relational Schema — relational model, constraints, the seven mapping steps.
6. Relational Operations — relational algebra; SELECT, PROJECT, RENAME, assignment.
7. Set Operations and Aggregation — union, intersection, difference, division, Cartesian product, join, aggregate functions, grouping.
8. Relational Algebra Hands On — COMPANY, Sailors and Pizza query sets.
9. SQL Overview and Data Definition — database languages, data types, DDL.
10. Constraints in SQL — key/entity/referential integrity, referential actions, ALTER, DROP, TRUNCATE, RENAME.
11. Database Modification — INSERT, SELECT, UPDATE, DELETE.

1 Data and Information

Data vs. Information

Data is raw, unorganised fact that must be processed to become meaningful — the individual puzzle pieces.

Information is data that has been processed, organised, structured or presented in a context so that it is useful. It is used by humans to make decisions and draw conclusions.

Dependency: information depends on data; data does not depend on information.

2 Database and DBMS

Database

A collection of *related* data representing some aspect of the real world, called the *mini-world* or *universe of discourse* (UoD).

Four qualifying properties:

- **Logically coherent** — organised with inherent meaning, not a random assortment.
- **Represents a mini-world** — captures some aspect of the real world.
- **Purpose-built** — designed, built and populated for a specific purpose and set of users/applications.
- **Variable in scale** — from a single application's small store to large enterprise databases.

Database Management System (DBMS)

A general-purpose software system that facilitates *defining, constructing, manipulating and sharing* databases among various users and applications.

A DBMS consists of a collection of interrelated data, a set of programs to access the data, and an environment that is convenient and efficient. Database systems manage data that is **highly valuable, relatively large and accessed by many users and applications at the same time**.

2.1 The four functions (with the student-information example)

Function	What it means	Student-information database
Define	Specify data types, structures and constraints; ensures consistency and integrity.	<code>Students(Student_ID num, Name text, Email text, DOB date, Major text); Courses(Course_ID, Course_Name, Credits, Instructor).</code>
Construct	Store the data on a storage medium controlled by the DBMS.	The database is stored on a server with adequate capacity and performance.
Manipulate	Query, update and generate reports, while enforcing constraints.	“Retrieve all students enrolled in CS courses”; add students; update grades.
Share	Allow multiple users and programs simultaneous access, subject to permissions.	Faculty see their courses' students; administrators update and maintain the whole database.

3 Why Study Databases

Databases let us	Mechanism
Store large amounts of data	file structure, disk management
Understand the data	data models
Keep data secure	security, recovery
Find, use and manipulate data	query languages, concurrency control, analysis tools
Get accurate information	built-in constraints and checks
Maintain data integrity	accuracy and consistency guarantees

4 Database Applications

Domain	Typical uses	Example
Enterprises	CRM, supply-chain management, human resources, financial systems	Salesforce CRM
Manufacturing	production tracking, supply chain, product-lifecycle management	Toyota (JIT)
Banking & finance	account management, loan processing, fraud detection	JPMorgan Chase
Web services	user-data management, content management, analytics	Netflix
Document DBs	semi-structured JSON/BSON/XML storage, flexible schema, horizontal scaling; CMS, catalogues, data lakes	MongoDB / NYT
Navigation	location data, real-time traffic, route planning, integration with other services	Google Maps

5 How Databases are Used: OLTP and OLAP

OLTP (transaction processing)

Large number of users performing *small* retrievals and updates.

Banking, universities, airlines — most database applications.

OLAP / data analytics

Processing data to draw conclusions and build predictive models.

Loan approval, targeted advertising, manufacturing decisions. Data mining combines AI and statistical techniques over large databases.

6 View of Data

A database system is a collection of interrelated data plus programs to access and modify it; its major purpose is to give users an **abstract view** of the data.

Data model and data abstraction

Data model: a collection of conceptual tools for describing data, data relationships, data semantics and constraints.

Data abstraction: hiding the complexity of the data structures used to represent data, through several levels of abstraction.

The structure of a database is defined by its data model.

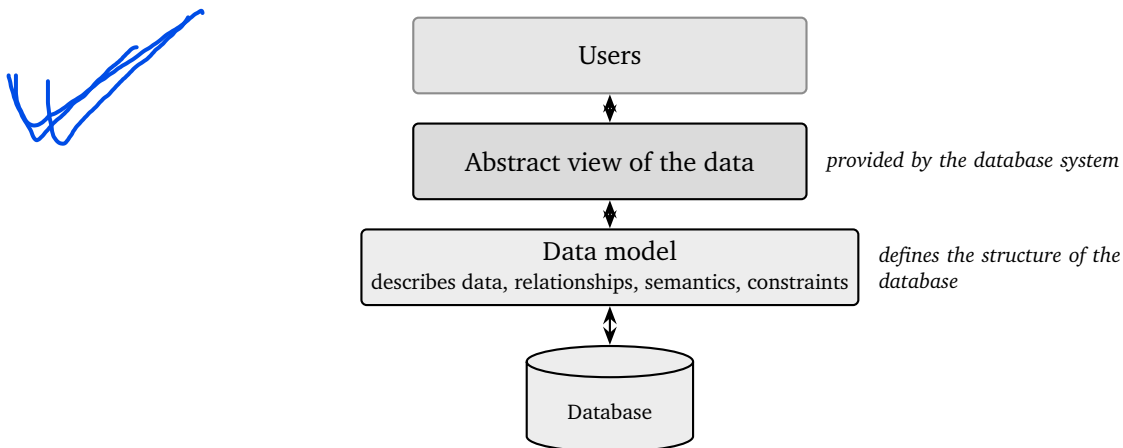


Fig 1.1 Users interact with an abstraction; the data model defines the underlying structure.

Chapter 2 — Purpose of Database Systems

File-processing Systems, their Drawbacks and the Database Solution

1 File-processing Systems

File-based system

Information is kept in permanent operating-system files. Each application program is written for a specific set of files, on request of the users, and new programs are added as needs arise.

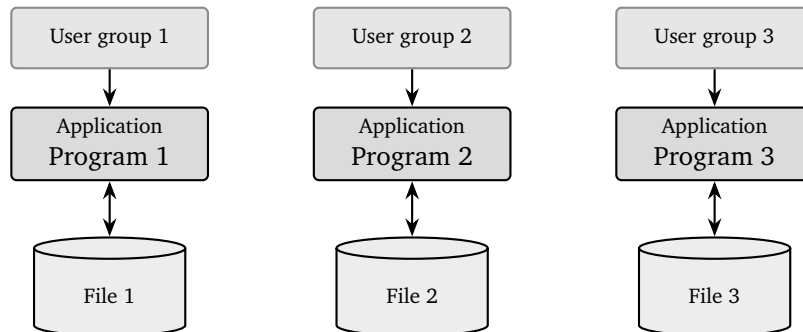


Fig 2.1 Each program owns its own files: nothing is shared, and every new requirement means a new file and a new program.

University example. Information is needed about Instructors, Students, Departments and Courses offered, with programs for: adding students/instructors/courses; registering students for courses; assigning grades, computing GPA and generating transcripts. Adding a *new major* then requires (i) creating a new department and new permanent files, and (ii) writing new application programs for registration, grading and transcripts.

Situations that expose the weakness

- **Double-major student** — how many times are the details stored?
- **Updating** — does an address change reach every departmental file?
- **Retrieval** — can we find all students in one postal code without writing a new program?
- **Format consistency** — files written in different formats are hard to access together.
- **New constraints** — can they be added without editing every program?

2 The Seven Drawbacks and their Database Solutions

	Drawback (with the lecture's example)	Database solution
1	Data redundancy & inconsistency. A double-major student's details are duplicated per department: wasted storage and disagreement after a change.	Centralised store minimises redundancy; a change made in one place keeps data consistent.
2	Difficulty accessing data. To list students in a postal code you must sift manually or write a special program.	Central store for fast retrieval; a powerful query language answers new queries; efficient search and filtering.
3	Data isolation. Data sits in several files in different formats; a file-based system serialises access by locking a whole file.	Many transactions may access the same data without interfering, each seeing a consistent view.
4	Integrity problems. "Account balance > 0" must be coded into every program; a new constraint means editing them all.	Constraints are declared in the schema and enforced automatically; adding one means updating only the schema.
5	Atomicity problems. Transfer of 5000 rupees from A to B may deduct from A and then fail before crediting B.	Atomicity: all steps or none; automatic rollback on failure keeps the state consistent.
6	Concurrent-access anomalies. Two clerks both read a balance of \$10,000 and deduct \$500 and \$100; the last writer wins, so the balance becomes \$9,500 or \$9,900 instead of \$9,400.	Transactions ensure atomicity; locking prevents conflicting changes; the DBMS maintains consistency and integrity.
7	Security problems. Without central security, payroll staff can read and change academic records.	The DBA defines access controls and permissions, enforced at database level: authentication, encryption, auditing/logging, central security management.

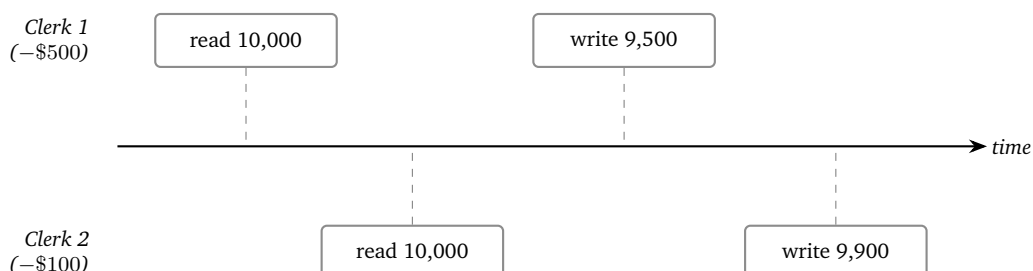


Fig 2.2 The lost-update anomaly: whichever clerk writes last determines the surviving balance.

3 Characteristics of the Database Approach

- **Self-describing nature.** The system holds the database *and* a complete definition of its structure and constraints (metadata) in the DBMS catalog.
- **Insulation between programs and data (program–data independence).** File structure is stored in the catalog separately from the access programs.
- **Support for multiple views.** Different users may see different subsets, or virtual data derived but not stored.
- **Sharing and multi-user transaction processing.** Concurrent access is controlled so that the results of changes are correct.

4 Advantages of Database Systems

- Controlling redundancy.
- Sharing data among users.
- Restricting unauthorised access.
- Persistent storage for program objects.
- Storage structures and search techniques for efficient query processing.
- Backup and recovery.
- Multiple interfaces for different classes of user.
- Representing complex relationships among data.
- Enforcing integrity constraints.
- Deductive/active rules and triggers to draw inferences and take actions.
- Enforcing standards.
- Reduced application-development time.
- Flexibility to change data structures.
- Availability of current information.
- Economies of scale.

5 Case Study — Identify the Problem

	Scenario	Problem type
1	Two customers withdraw from the same account at once; lacking concurrency control the account is overdrawn.	Concurrent access anomalies
2	Each department keeps its own spreadsheet of customers; addresses and contact details disagree.	Data redundancy and inconsistency
3	Inventory and order systems are separate and not updated in real time, so stock is oversold.	Data isolation
4	Grades are entered manually with no automatic check; an invalid grade 'x' is accepted.	Integrity problems

Chapter 3 — Database Design

Data Abstraction, Data Independence, Instances and Schema, Users and Architectures

1 Data Abstraction — The Three-Schema Architecture

Complex data structures are needed for efficiency, but most users are not trained to handle them, so the developer hides the complexity behind levels of abstraction.

Level	What it describes
Physical	The lowest level: how the data is actually stored in the computer.
Logical	What data is stored and how the pieces are related; describes the whole database in terms of a small number of relatively simple structures.
View	The highest level; each user sees only the part of the database relevant to them, according to their needs and permissions.

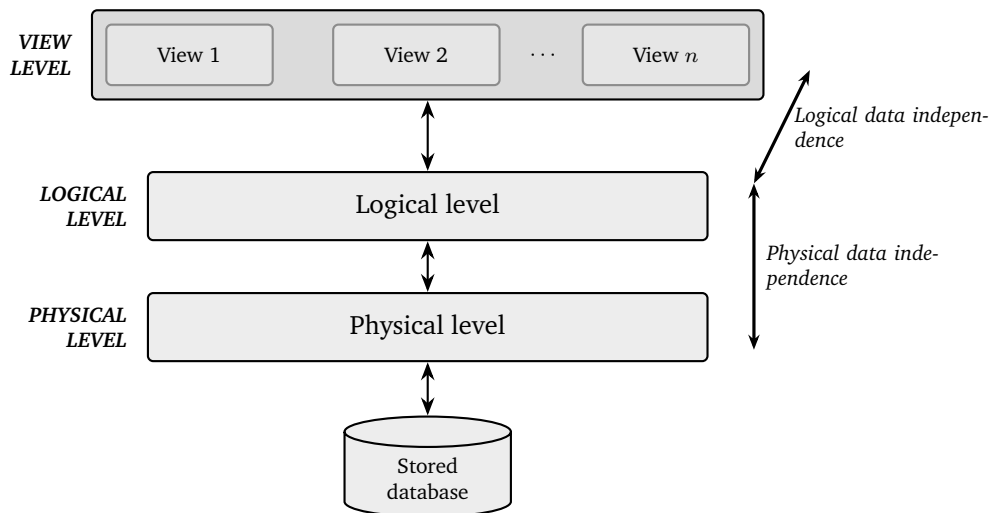


Fig 3.1 The three levels of abstraction, with the two forms of data independence between adjacent levels.

A user at the logical level need not know how simple structures are implemented physically; a user at the view level need not know the logical-level implementation.

2 Data Independence

Physical data independence

The ability to modify the *physical* schema without altering the conceptual/logical schema (done for optimisation). Changes may include new storage devices, different storage data structures, altered indexes or alternative file organisations — e.g. moving from sequential to random-access files.

Logical data independence

The ability to modify the *logical* schema without affecting the external schema or application programs — e.g. inserting or deleting attributes, or altering tables, entities and relationships.

Change	Without independence	With independence
Migrating the storage system / optimising layout	Minor storage or hardware changes break queries, applications and interfaces; extensive rework, higher maintenance cost, longer development cycles.	The DBA changes storage without touching the logical schema; queries and applications keep working; easier maintenance, upgrades and tuning.
Reorganising departments / adding attributes to an entity	Every change propagates to all applications; developers must modify affected code; even minor changes cause disruption, downtime and risk of error.	Designers modify the logical schema without impacting existing applications; the design adapts easily to new requirements.

2.1 The university database at the three levels

Physical	How student records, course information and department data are stored on disk or in memory.
Logical	Entities “Department”, “Student”, “Course” with attributes (department name, student ID, course code) and relationships (“students belong to departments”, “courses are taken by students”).
View	A faculty member sees only their courses and the students enrolled; a student sees only their registered courses and grades.

3 Instances and Schema

Schema and instance

The overall *design* of the database is the **database schema**. The collection of information stored at a particular moment is an **instance** (also called the **database state** or snapshot).

Programming analogy: **schema** $\Rightarrow$ variable declarations with their type definitions; **instance** $\Rightarrow$ the value of the variable at a point in time.

- Actual data changes frequently; in a given state each schema construct has its own current set of instances.
- Many states can correspond to one schema.
- When a new database is defined, only the schema is given to the DBMS and the state is **empty**; the **initial state** arrives when the database is first populated; every update produces another state, and at any moment there is a **current state**.

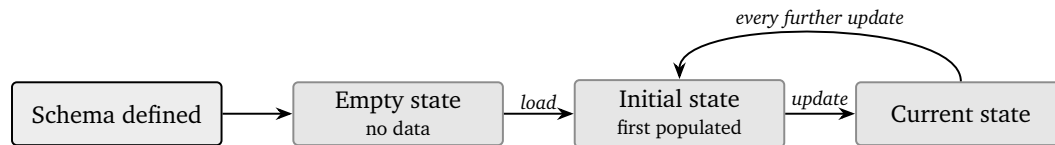


Fig 3.2 One schema definition, followed by a succession of states.

4 Database Users

Naïve (parametric)	No knowledge of databases; use applications through a predefined interface — e.g. booking a movie ticket.
Application programmers	Back-end developers who write application programs — connect an app to the database, write queries to display products, manage registrations and orders, handle errors, secure data.
Sophisticated users	Deep understanding of the DBMS; use DDL and DML directly — data engineers and developers fetching, deleting, updating and inserting via SQL.
Database administrator	Central control of the data and the programs that access it; defines the schema and controls the database.

4.1 Functions of the DBA

1. **Schema definition** — creates the original schema using DDL.
2. **Storage structure and access-method definition** — parameters for physical organisation and index creation.
3. **Schema and physical-organisation modification** — changes to reflect needs or improve performance.
4. **Granting authorisation** — regulates access by granting types of authorisation.
5. **Routine maintenance** — periodic backups, disk-space availability, performance monitoring.

5 Typical DBMS Component Modules

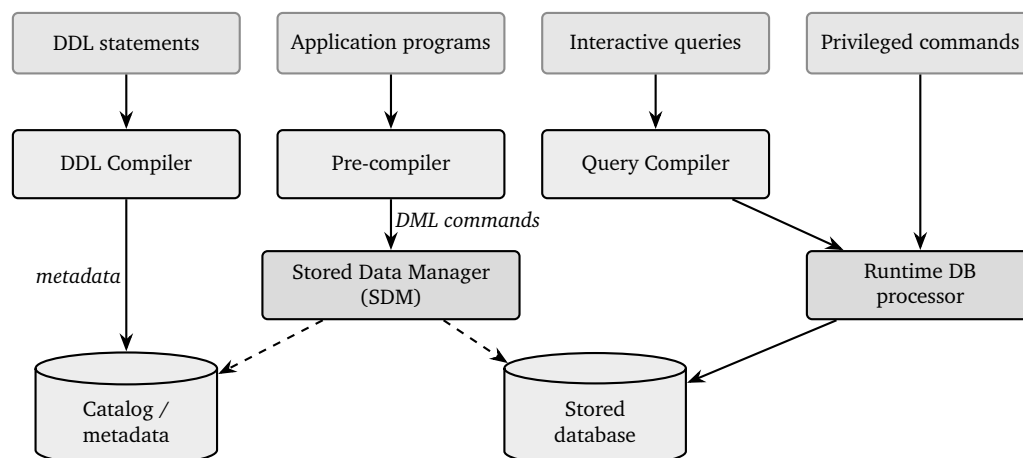


Fig 3.3 DBMS component modules; dotted lines show the Stored Data Manager's control of disk access.

- **Stored Data Manager (SDM)** — controls access to DBMS information stored on disk.
- **DDL compiler** — processes schema definitions and stores metadata in the catalog.
- **Query compiler** — parses and compiles high-level queries and calls the runtime processor.
- **Pre-compiler** — extracts DML commands from the application program.
- **Runtime database processor** — executes privileged commands, queries and canned transactions.

5.1 Database system utilities

- **Loading** — loads existing data files, with conversion from the old format.
- **Backup** — periodic incremental or total backups.
- **Storage reorganisation** — reorganises file structures and creates new access paths.
- **Performance monitoring** — reports usage statistics to the DBA.

6 Database Architectures

Centralized	Single location; all data on a central server. Shared memory for multiple CPUs; CPU, memory and storage centralised and easy to manage. Suits small/medium applications and legacy systems.
-------------	---

Client-server	A client (browser, mobile app, desktop application) requests data; a server (MySQL, Oracle, Postgres) manages the database, processes requests and handles storage and retrieval.
Parallel	Runs on a cluster of machines; many cores, shared memory and shared disk; better scalability and processing power. Used for data warehousing, real-time analytics, high-throughput transactions (e.g. NASA's satellite data on HPC systems).
Distributed	Data stored across multiple physical locations, in one data centre or geographically spread. Advantages: scalability, fault tolerance through replication, and performance through distributing queries.

6.1 Two-tier client-server

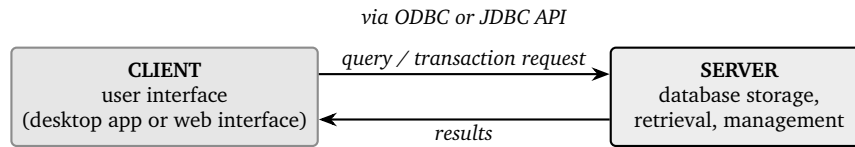


Fig 3.4 The client sends query/transaction requests through ODBC or JDBC; the server returns results.

- **Clients** provide interfaces to server resources through a client software module; may be diskless machines, PCs or workstations; connected over a network.
- **Servers** provide query and transaction services; relational DBMS servers are called SQL servers, query servers or transaction servers.
- Applications use an API — **ODBC** (Open Database Connectivity) or **JDBC** (for Java) — and both sides must install the matching module.

6.2 Three-tier client-server

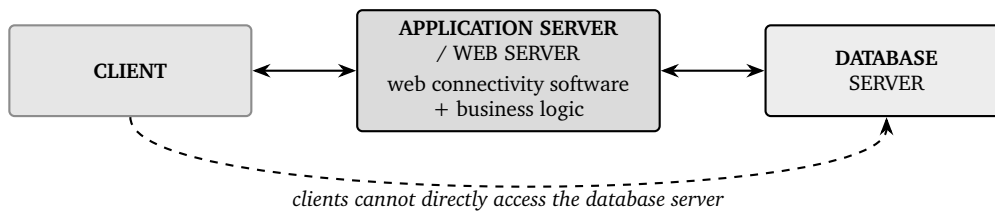


Fig 3.5 The middle tier is the only route to the database server, which is what makes it more secure.

- Common for web applications. The middle tier (application/web server) holds the web connectivity software and business logic, and acts as a conduit for partially processed data.
- Security: the database server is reachable only through the middle tier; clients cannot access it directly.

Chapter 4 — Entity-Relationship Model

Entities, Attributes, Relationships, Weak Entities, Cardinality and Participation

1 Database Design Phases and Why the E-R Model

Design proceeds through: *characterising user data needs* → *functional requirements specification* → *choosing a data model and translating requirements* → *logical-design phase* → *review and refinement of schema* → *physical-design phase*. E-R diagrams are used because they (i) give a graphical representation of the logical structure using symbols for entities, relationships and attributes; (ii) are simple and clear, hence widely used; (iii) expose redundancies and inconsistencies *before* implementation, avoiding costly later modification; and (iv) serve as documentation of the database structure.

The COMPANY requirements (used throughout Chapters 4 and 5)

- The company is organised into **DEPARTMENTS**. Each has a name, number and an **employee who manages it; the manager's start date is recorded. A department may have several locations.**
- Each department controls a number of **PROJECTS**. Each project has a unique name and number and is at a single location.
- For each **EMPLOYEE** the database stores SSN, address, salary, sex and birthdate. Each employee works for one department but may work on several projects, and the **hours per week** on each project are recorded. The direct **supervisor** of each employee is tracked.
- Each employee may have a number of **DEPENDENTS**, each with name, sex, birthdate and relationship to the employee.

2 Entities

Entity, entity set

An **entity** is a specific thing or object in the mini-world that is represented in the database. It may have *physical existence* (car, person, employee, house) or *conceptual existence* (a company, a job, a university course, a project).

An **entity set** (entity collection) is a group of entities of the same kind sharing common attributes — e.g. “instructor” is the set of all university instructors.

- Each entity has attributes; unique property values identify each entity (e.g. `student_id = PES001`, `course_id`).
- If E_1 is an entity of type *Student*, the set of all students is the entity set.
- Two kinds of entity: **strong** and **weak** (Section 7).

3 Attributes

Attributes are the qualities or characteristics of an entity; they signify the type of information stored, and every entity has its own values. An attribute such as `ID` acts as a unique identifier, preventing confusion when two instructors share a name. (Government-issued IDs were once used but raise security and privacy concerns, so enterprises define their own identifiers.)

Domain / value set

For each attribute there is a set of permitted values, called the **domain** or **value set**. E.g. ages 16–70 for employees; `Name` as strings of alphabetic characters separated by blanks.

Formally an attribute A of entity set E with value set V is a function $A : E \rightarrow P(V)$, the power set of V .

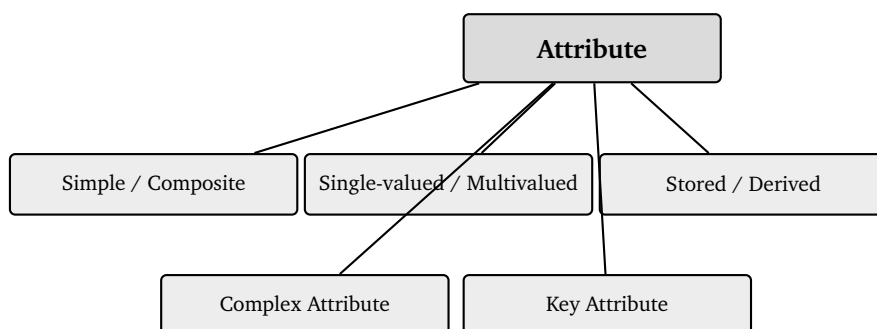


Fig 4.1 The attribute characterisations used in the E-R model.

3.1 Simple vs. composite

Simple and composite attributes

Attributes that cannot be subdivided are **simple**; attributes that can be split into components are **composite**.

- Simple: student/instructor `ID`, roll number, phone number, email id.
- Composite: `address` → house number, street, city, state, country, pin code; `name` → first, middle, last.

Why it matters. If “123 Main Street, Yeshwanthpur, Bangalore, 560063” is stored in a single attribute, students cannot be grouped by postal code for bus allotment, because the postal code is not an attribute of its own. Splitting the attribute solves this. Composite attributes are a good choice when a user sometimes wants the whole attribute and sometimes only a component.

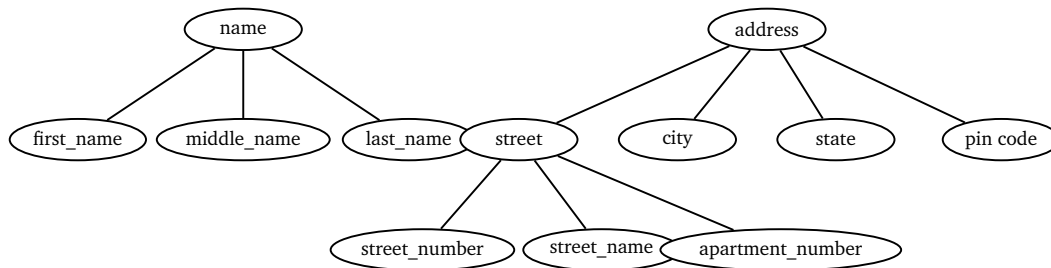


Fig 4.2 Composite attributes form a hierarchy: *street* of *address* divides again into street number, street name and apartment number.

3.2 Single-valued vs. multivalued

Single-valued and multivalued attributes

A **single-valued** attribute takes one value per entity instance (name, age). A **multivalued** attribute may take more than one value per instance (an instructor's phone numbers, mail ids). It is drawn as a *double ellipse*.

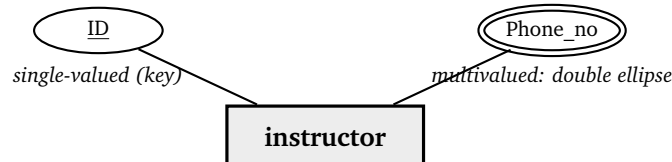


Fig 4.3 *Phone_no* is multivalued (double ellipse); *ID* is single-valued and is the key.

3.3 Complex attributes

Complex attribute

An attribute formed by *nesting* composite and multivalued attributes. Rarely used.

PreviousDegrees of a STUDENT

{PreviousDegrees(College, Year, Degree, Field)} — multiple values may exist (multivalued), and each has four subcomponents (composite); hence complex.

3.4 Stored vs. derived

Stored and derived attributes

Stored attributes are physically held in the database (*Date_of_Birth*). A **derived** attribute can be computed from others (*Age* from *Date_of_Birth* and the current date) and is drawn with a *dashed oval*.

In *Instructor*(*ID*, *Name*, *Department*, *Date_of_Birth*, *Age*) the attribute *Age* need not be stored at all.

3.5 Key attributes

Key attribute

An attribute of an entity type for which each entity must have a *unique* value — e.g. *SSN* of *Employee*, *SRN* of *Student*.

- A key may be composite: *VehicleTagNumber* = (*Number*, *State*) is a key of *CAR*.
- An entity type may have more than one key: *CAR* has *VIN* and *VehicleTagNumber*.
- In an E-R diagram *each* key is underlined — unlike a relational schema, where only the primary key is underlined.

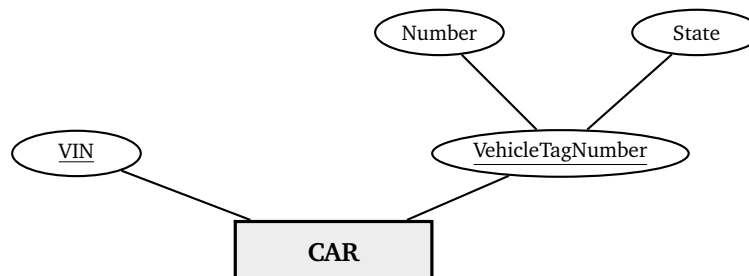


Fig 4.4 An entity type with two keys, one of them composite; both are underlined.

4 Null Values

An attribute takes a **null** value when the entity has no value for it (e.g. a student with no middle name). Null may mean:

- **Not applicable** — the value does not exist for this entity.
- **Unknown**, which splits into **missing** (the value exists but is not recorded) and **not known** (we do not know whether the value exists).

For an apartment-number attribute the three readings are: the address has no apartment number; it has one but we do not know it; we do not know whether it has one.

5 Relationships

Relationship and relationship set

A **relationship** signifies a connection between entities — *advisor* links instructor Katz to student Shankar. A **relationship set** contains relationships of the same type; the *advisor* set links students and their advisors.

A **relationship instance** associates named entities: instructor Katz (ID 45565) advising student Shankar (ID 12345). In an E-R diagram a relationship set is drawn as a *diamond* joined by lines to the participating entity rectangles.

5.1 Recursive relationships

The function an entity plays in a relationship is its **role**. Roles are usually implicit, but when the *same* entity set participates more than once — a **recursive relationship set** — explicit role names are necessary; they label the lines joining the diamond to the rectangle.

Course prerequisites

Course C_2 is a prerequisite for course C_1 ; the relationship set *prereq* is modelled by ordered pairs of *course* entities, with roles *course_id* and *prereq_id*. Another example: Employee–Supervisor.

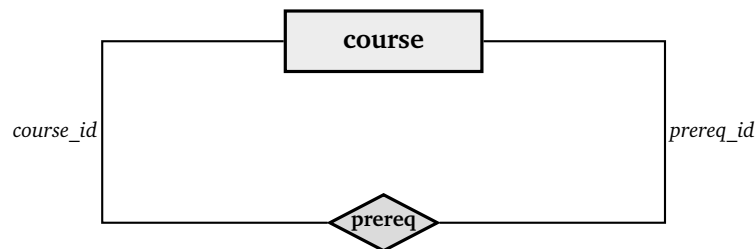


Fig 4.5 A recursive relationship: one entity set participating twice in two roles.

5.2 Degree, and ternary relationships

The **degree** of a relationship set is the number of entity sets that participate: binary = 2 (most relationships), ternary = 3.

Research projects — a ternary relationship

Entity sets *instructor*, *student* and *project*: a project has many students and many instructors, and each student on a project has an instructor guiding them. The ternary set *proj_guide* records which instructor guides which student on which project — a student may have different guides on different projects, which no binary relationship between students and instructors can capture.

5.3 Relationship (descriptive) attributes

A relationship may itself carry attributes, called **descriptive attributes**, drawn as an oval attached to the diamond. *Where does grade belong?* Not in *Student* nor in *Course*: a grade exists only because a student enrolled in a course, so *grade* is an attribute of the relationship *enroll*.

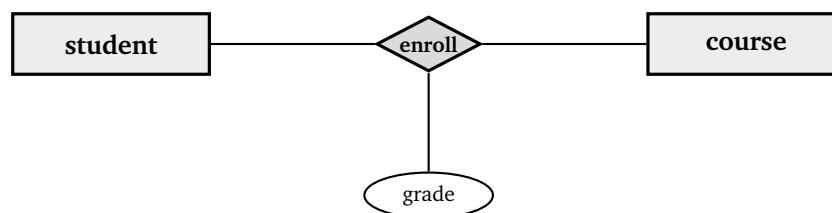


Fig 4.6 *grade* is a descriptive attribute of *enroll*.

6 Worked Examples — Modelling an Entity

Example 1 — Instructor

Unique ID; name split into first/middle/last; address split into street, city, state, zip, and *street* split again into *street_no*, *street_name*, *apt_number*; *Date_of_Birth* stored and *age* derived; multiple *phone_numbers*; exactly one *department*.

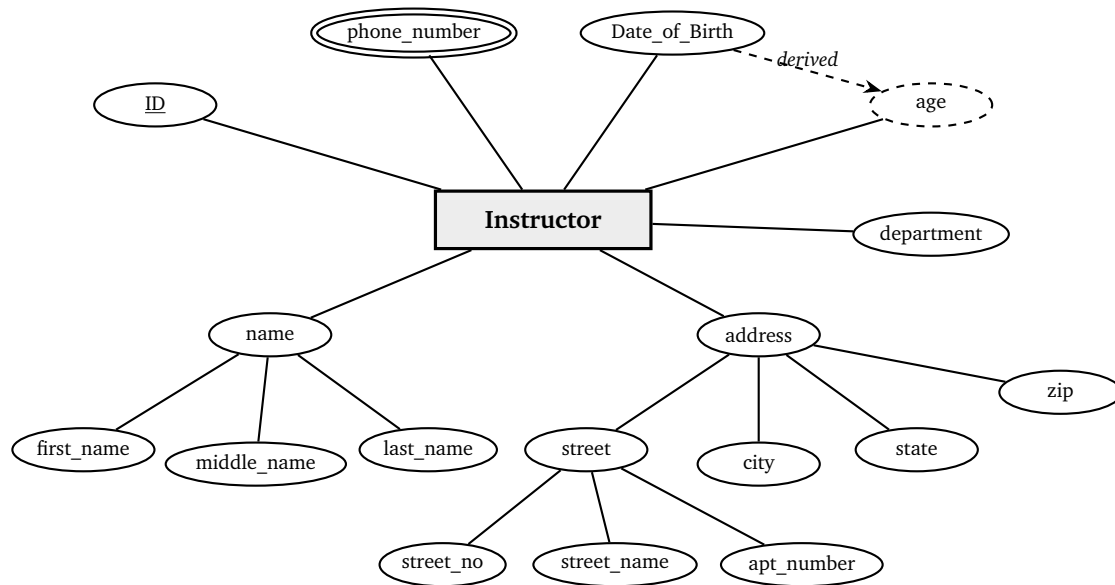


Fig 4.7 The Instructor entity: a key attribute, a multivalued attribute, a derived attribute and two levels of composite attribute.

Example 2 — Environmental Sensor (Smart City)

Each sensor is uniquely identified (Sensor_ID), has a Type (temperature, humidity, pollution), a Location and a Last_Calibration_Date, and collects multiple Data readings, each with Date-Time and Value — a composite and multivalued attribute.

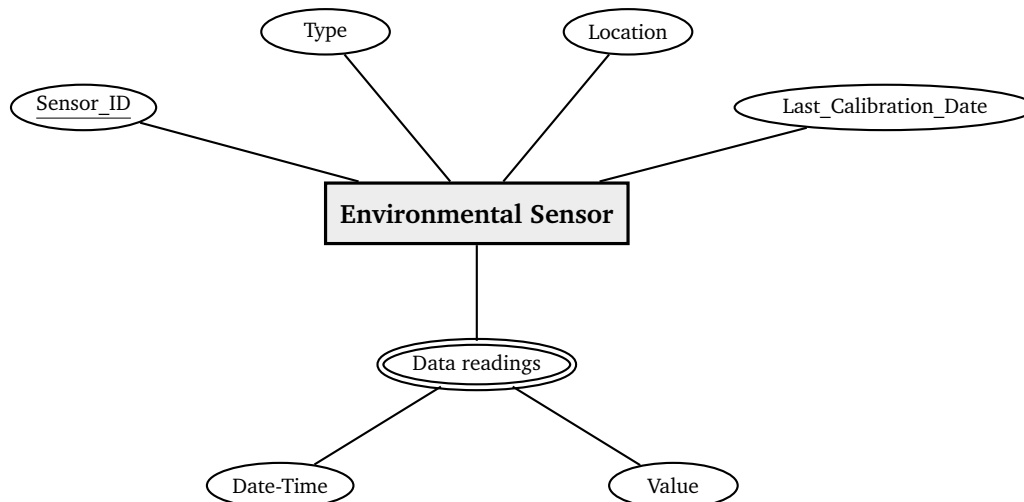


Fig 4.8 The Environmental Sensor with a key attribute and a composite multivalued attribute.

7 Weak Entities

Weak entity

An entity whose existence depends on another entity, called its **identifying entity**; an entity that is not weak is a **strong** entity.

- Every weak entity must be associated with an identifying (strong) entity — it is **existence dependent** on it.
- The relationship joining them is the **identifying relationship**.
- Key of the weak entity = key of the identifying entity + **discriminator** (partial key) of the weak entity.
- A weak entity set must have **total participation** in its identifying relationship set.
- A weak entity may take part in other relationships too, and may even have *more than one* identifying entity — then its key is the union of the owners' primary keys plus its discriminator.

Course-Section

Course(Course_ID, title, dept_name, credits) and Section(Course_ID, sec_ID, semester, year, building, room_number, time_slot_id).
 Weak entity: *Section*. Identifying entity: *Course*. Identifying relationship: *sec_course*. Discriminator: *sec_id*. Primary key: {course_id, sec_id, semester, year}.

Notation. A weak entity is a *double rectangle*; its discriminator is underlined with a *dashed* line; the identifying relationship is a *double diamond*; total participation is a *double line*.

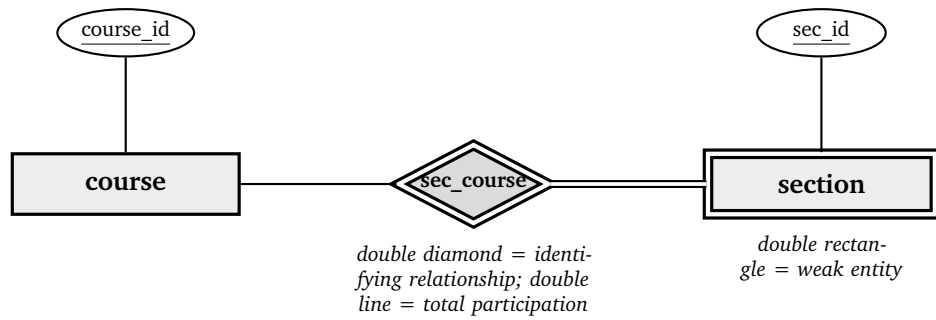


Fig 4.9 The three notational devices that mark a weak entity, its discriminator and its identifying relationship.

8 Cardinality Ratio

Mapping cardinality

Mapping cardinalities (cardinality ratios) express the number of entities to which another entity can be associated via a relationship set. They are most useful for binary relationship sets.

One-to-one (1:1)	An entity in <i>A</i> is associated with at most one entity in <i>B</i> , and an entity in <i>B</i> with at most one in <i>A</i> .
One-to-many (1:N)	An entity in <i>A</i> is associated with any number (zero or more) of entities in <i>B</i> ; an entity in <i>B</i> with at most one in <i>A</i> .
Many-to-one (N:1)	An entity in <i>A</i> is associated with at most one entity in <i>B</i> ; an entity in <i>B</i> with any number in <i>A</i> .
Many-to-many (M:N)	Any number on both sides.

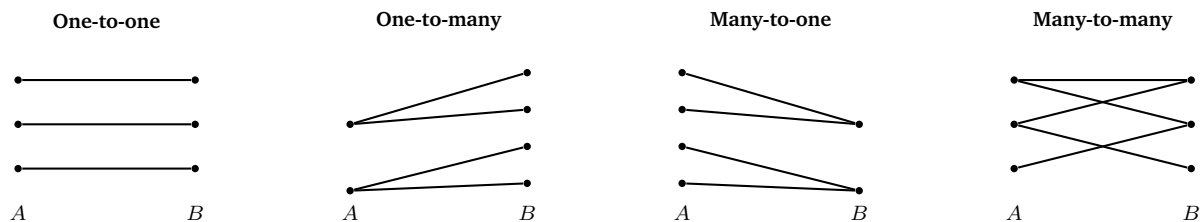


Fig 4.10 The four mapping cardinalities.

Cardinality is shown by drawing an undirected line from the relationship set to both entity sets and labelling it. *Advisor*: if a student may be advised by several instructors the set is M:N; if the university allows only one advisor per student while an instructor advises many, it is 1:N.

Company example	Ratio	Meaning
MANAGES	1:1	An employee manages at most one department; a department has at most one manager.
WORKS_FOR	1:N	A department has any number of employees; an employee works for at most one department.
WORKS_ON	M:N	An employee works on several projects; a project has several employees.

9 Participation Constraints

Total and partial participation

Participation of entity set *E* in relationship set *R* is **total** if every entity in *E* must participate in at least one relationship in *R* (drawn with a *double line*); it is **partial** if some entities of *E* need not participate (single line).

Advisor: if every student must have an advisor, student's participation is total; an instructor need not advise anyone, so instructor's participation is partial.

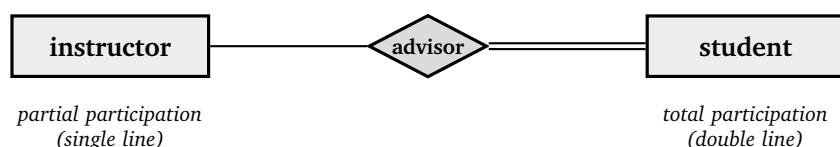


Fig 4.11 The double line indicates that a student must have an advisor.

10 The Min-Max Notation

The problem that motivates min-max

Every student must enrol in a minimum of 2 and a maximum of 5 courses. Plain cardinality cannot express this.

A line may carry a minimum and maximum cardinality written (l, h) :

minimum value = 1	total participation of that entity set in the relationship set
maximum value = 1	the entity participates in at most one relationship
maximum value = <i>N</i>	no limit

So *Student* (2, 5) *Enrolls* (0, N) *Course* expresses the rule above.

Min-Max Notation and Chen Notation Compared

Min-Max [Entity 1]	Chen Notation	Min-Max [Entity 2]
(0,1)	1:1	(0,1)
(0,N)	1:n	(0,1)
(0,N)	1:n + total part.	(1,1)
(0,N)	n:m	(0,N)
(1,1)	total part. + 1:1	(0,1)
(1,N)	total part. + 1:n	(0,1)
(1,1)	total part. + 1:1 + total part.	(1,1)
(1,N)	total part. + 1:n + total part.	(1,1)
(1,N)	total part. + n:m	(0,N)
(1,N)	total part. + n:m + total part.	(1,N)

Fig 4.12 Min-max pairs and the equivalent Chen notation.

11 Summary of E-R Notation

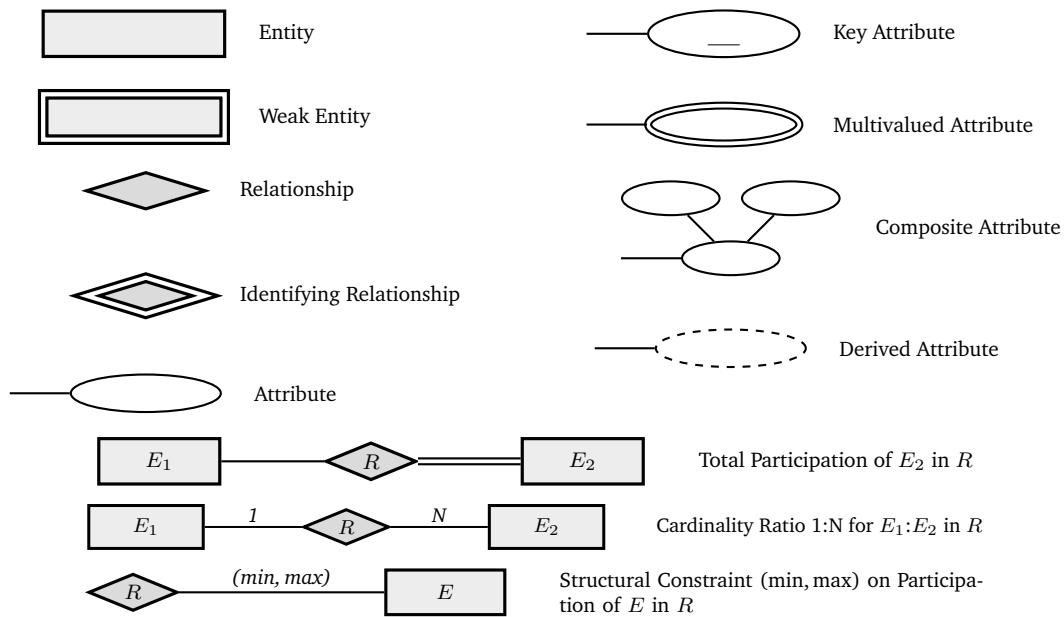


Fig 4.13 The complete E-R notation: entity, weak entity, relationship, identifying relationship, the attribute types, total participation, cardinality ratio and structural (min, max) constraints.

12 Worked Examples — Smart City

Sensor and Infrastructure

Infrastructure(Infrastructure_ID, Type, Location, Installed_Date, Operational_Status). Each sensor is installed on *exactly one* infrastructure element (total participation on the sensor side), and each infrastructure element hosts *zero or more* sensors (partial, N side). Leads_To models a recursive relationship inside Infrastructure (one road leads to another).

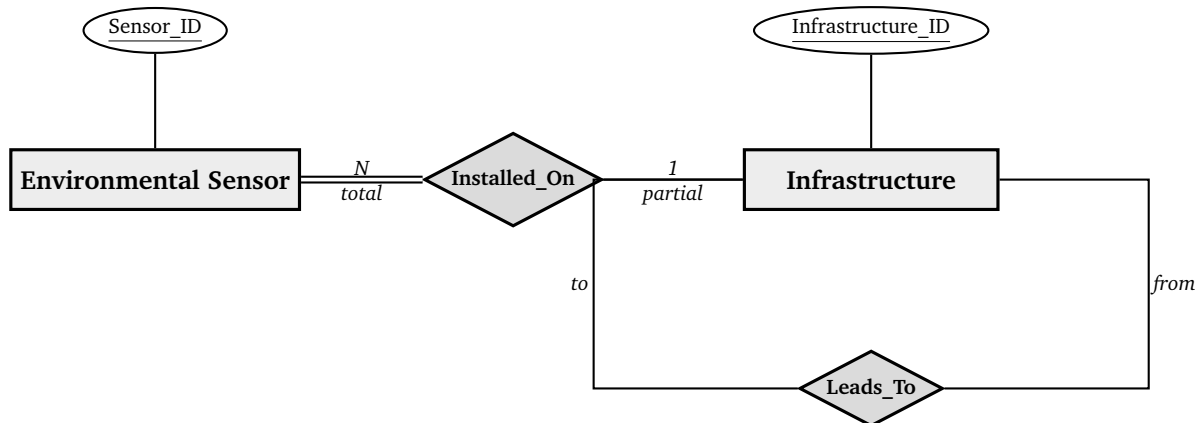


Fig 4.14 Sensor-Infrastructure with cardinality and participation, plus the recursive Leads_To.

Maintenance Requests

Citizens report issues (broken streetlights, potholes, faulty signals) with a *description*, *date* and *status*. A citizen may raise many requests; a request may be reported by several citizens; each request belongs to exactly one infrastructure

element and *cannot exist* without both a citizen and an infrastructure element — so Maintenance Request is a **weak entity** identified by both, drawn with double diamonds.

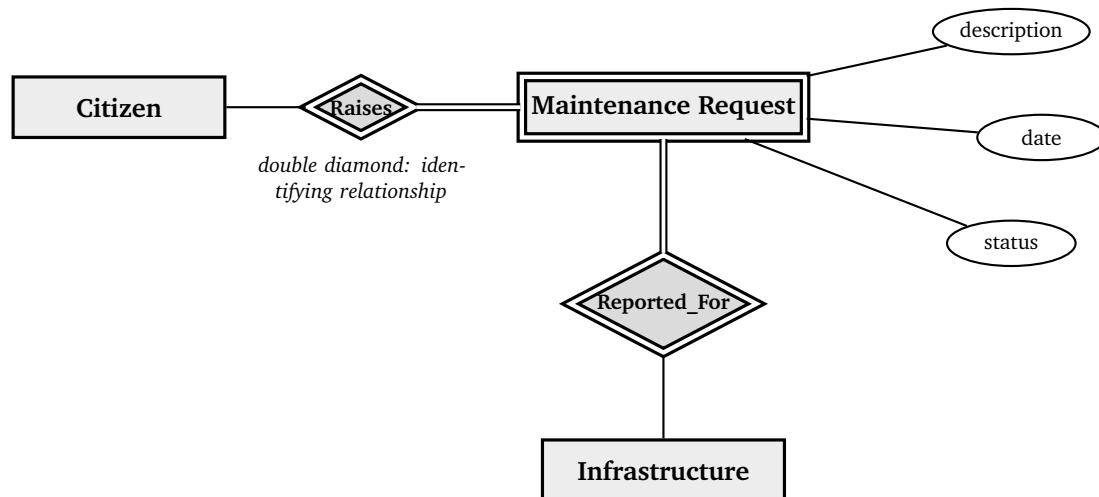


Fig 4.15 Maintenance Request as a weak entity dependent on both Citizen and Infrastructure.

Chapter 5 — Converting an E-R Diagram to a Relational Schema

Relational Model, Constraints and the ER-to-Relational Mapping Algorithm

1 The Relational Model

The relational model represents the database as a collection of **relations**; informally each relation is a table of values. Each row holds facts about one real-world entity or relationship; the table and column names give the values their meaning.

Formal term	Informal term
Relation	Table
Tuple	Row
Attribute (attribute name)	Column (column header)

1.1 Formal definitions

Domain, schema, state

Domain: a set of permitted values with a logical definition and a data type/format — e.g. `USA_phone_numbers` formatted (ddd) ddd-dddd; dates as `yyyy-mm-dd`.

Relation schema: $R(A_1, A_2, \dots, A_n)$ where R is the relation name and A_i are its attributes, each with a domain. Example: `CUSTOMER(Cust-id, Cust-name, Address, Phone#)`, where `Cust-id` has domain “6-digit numbers”.

Relation state: r of schema $R(A_1, \dots, A_n)$ is a set of n -tuples $r = \{t_1, t_2, \dots, t_m\}$. A tuple is an ordered set of values. The schema R is the *intension*; the state r is the *extension*.

Key of a relation. The data item (or set of items) whose value uniquely identifies a row — e.g. `SSN` in `STUDENT`. A row-id or sequential number assigned for this purpose is an **artificial** or **surrogate** key.

2 Constraints in the Relational Model

Inherent / implicit	Come from the data model itself — e.g. the relational model does not allow a list as an attribute value.
Schema-based / explicit	Expressed in the schema using the model’s facilities — e.g. the maximum cardinality ratio in the ER model.
Application-based / semantic	Beyond the expressive power of the model; must be specified and enforced by application programs.

2.1 The five schema-based constraints

1. Domain constraint. Every value in a tuple must come from the domain of its attribute (or be null, where allowed).

2. Key constraints

Super key: a set of attributes SK of R such that no two tuples in any valid state $r(R)$ have the same value for SK : for distinct tuples t_1, t_2 , $t_1[SK] \neq t_2[SK]$.

Key: a *minimal* super key — removing any attribute from K leaves a set K' that is no longer a super key. Every key is a super key, but not conversely.

Candidate key: each of the several keys a schema may have — e.g. `CAR` has registration number, engine serial number and licence number.

Primary key: one candidate key chosen (arbitrarily) to identify tuples; its attributes are underlined. `CAR(licenseNo, Reg#, SerialNo, Make, Model, Year)`.

3. Constraints on NULLs. Specifies whether NULL is permitted — e.g. `Name` of `STUDENT` is `NOT NULL`.

4. Entity integrity. The primary-key attributes PK of each relation may not be null in any tuple, since they identify tuples: $t[PK] \neq \text{null}$ for every t in $r(R)$.

5. Referential integrity — foreign key

A set of attributes FK in R_1 is a **foreign key** referencing R_2 if:

- the attributes of FK have the same domain(s) as the primary key PK of R_2 ; and
- a value of FK in a tuple t_1 of $r_1(R_1)$ either occurs as a PK value of some tuple t_2 in $r_2(R_2)$ — then $t_1[FK] = t_2[PK]$ and t_1 *references* t_2 — or is NULL.

R_1 is the *referencing* relation, R_2 the *referenced* relation.

3 Reducing an E-R Diagram to a Relational Schema

Both models are abstract, logical representations of the same organisation, so an E-R design can be transformed into a relational design: for each entity set and each relationship set there is a unique relation schema carrying the corresponding name.

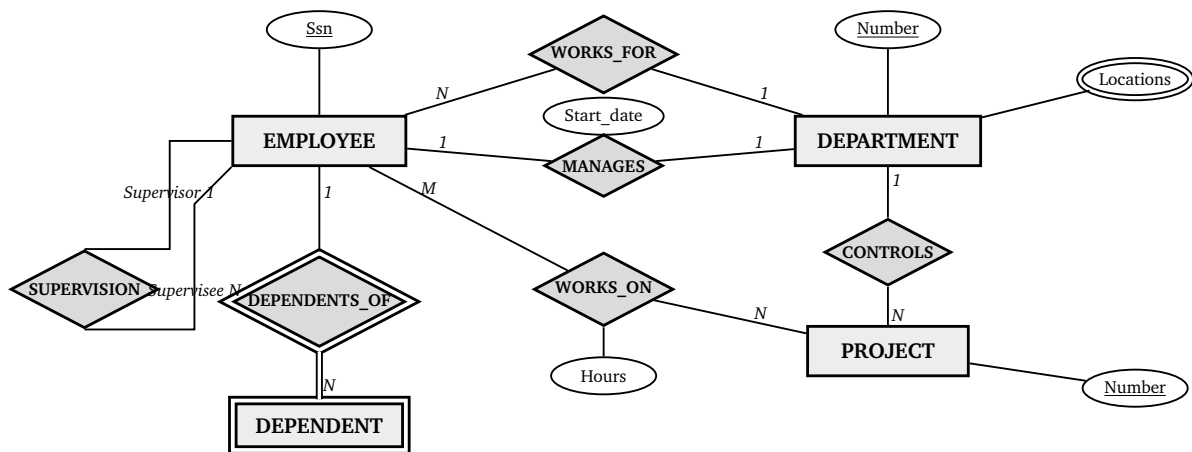


Fig 5.1 The COMPANY ER schema on which the seven mapping steps operate.

Step 1 — Regular (strong) entity types

- For each strong entity type E , create a relation R with all the *simple* attributes of E .
- Choose one key attribute of E as the primary key of R ; if it is composite, the set of simple attributes forming it together become the primary key.

Company: relations EMPLOYEE, DEPARTMENT, PROJECT with primary keys Ssn , $Dnumber$, $Pnumber$.

Step 2 — Weak entity types

- For each weak entity type W with owner E , create a relation R with all simple attributes (or simple components) of W .
- Include the primary key(s) of the owner relation(s) as **foreign key** attributes.
- Primary key of R = owner's primary key(s) + partial key of W .

Company: DEPENDENT includes EMPLOYEE's Ssn as a foreign key renamed $Essn$; its primary key is $\{Essn, Dependent_name\}$, since $Dependent_name$ is the partial key.

Step 3 — Binary 1:1 relationship types

Identify the relations S and T of the participating entity types; three approaches:

1. **Foreign key (2 relations).** Put the primary key of T into S as a foreign key. Prefer as S the entity type with *total* participation in R .
2. **Merged relation (1 relation).** Merge both entity types and the relationship into one relation — appropriate when both participations are total.
3. **Cross-reference / relationship relation (3 relations).** A third relation holds the primary keys of S and T .

Company: MANAGES is mapped by choosing DEPARTMENT as S , because its participation is total.

Step 4 — Binary 1:N relationship types

- Identify the relation S representing the entity type on the **N** side.
- Include in S , as a foreign key, the primary key of the relation T on the 1 side.
- Include any simple attributes of the relationship as attributes of S .

Company: WORKS_FOR, CONTROLS and SUPERVISION are 1:N. For WORKS_FOR, $Dnumber$ of DEPARTMENT goes into EMPLOYEE as Dno . (A relationship relation could be used instead, but rarely is.)

Step 5 — Binary M:N relationship types

- Create a new **relationship relation** S for R .
- Include the primary keys of both participating relations as foreign keys; *their combination* is the primary key of S .
- Include any simple attributes of the relationship.

Company: WORKS_ON has foreign keys $Essn$ and Pno (primary key $\{Essn, Pno\}$) plus the relationship attribute $Hours$.

Step 6 — Multivalued attributes

- For each multivalued attribute A , create a new relation R containing an attribute for A plus the primary key K of the owning relation as a foreign key.
- Primary key of R = $\{A, K\}$; if A is composite, include its simple components.

Company: DEPT_LOCATIONS holds $Dlocation$ (from $Locations$ of DEPARTMENT) and $Dnumber$; primary key $\{Dnumber, Dlocation\}$.

Step 7 — n -ary relationship types

For each n -ary relationship type with $n > 2$, create a relation S containing the primary keys of all participating relations as foreign keys, plus any simple attributes of the relationship.

The SUPPLY relationship

Ternary SUPPLY relates SUPPLIER, PART and PROJECT and carries *Quantity*; it maps to relation SUPPLY with primary key {SName, PartNo, ProjName}.

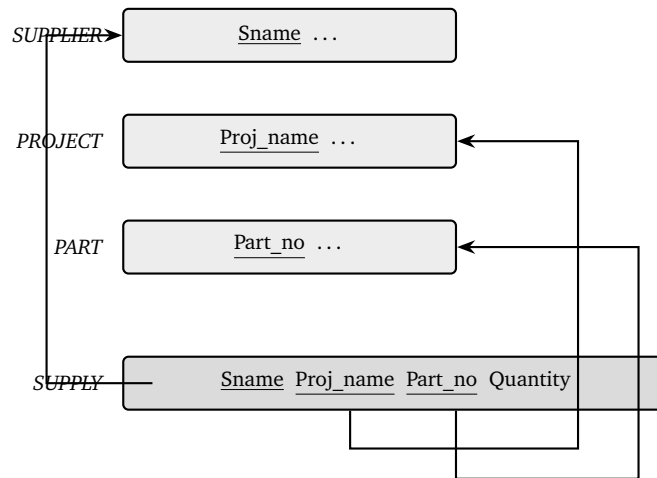


Fig 5.2 Mapping SUPPLY: three foreign keys whose combination is the primary key.

4 The Resulting COMPANY Relational Schema

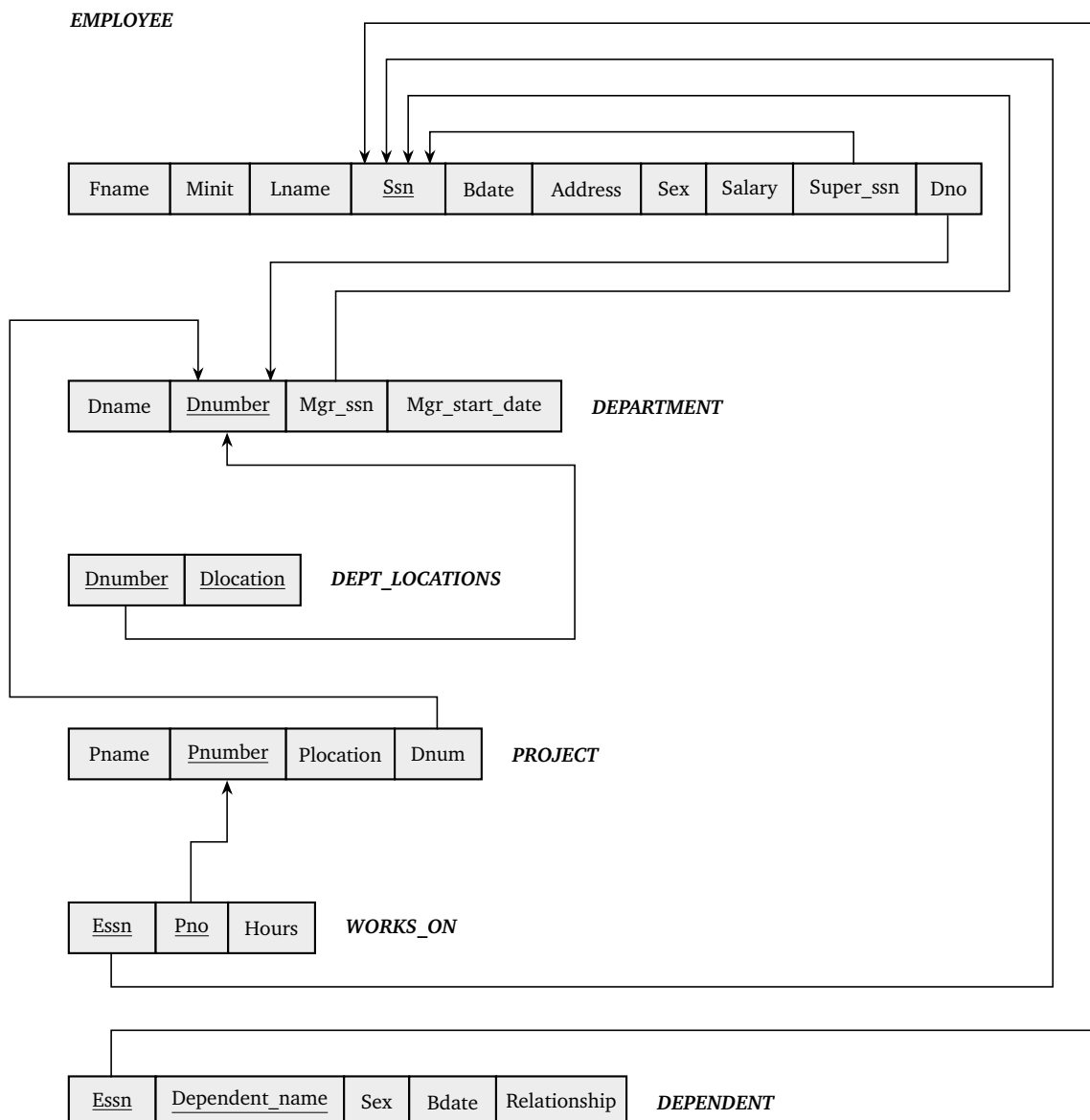


Fig 5.3 The COMPANY ER schema mapped to relations. Underlined attributes are primary keys; arrows run from a foreign key to the primary key it references.

5 Summary of Mapping Constructs

ER model	Relational model
Entity type	Entity relation
1:1 or 1:N relationship type	Foreign key (or relationship relation)
M:N relationship type	Relationship relation and two foreign keys
n -ary relationship type	Relationship relation and n foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

6 Worked Example — A Bank Database

The scenario

Each bank has multiple branches; each branch has multiple accounts and loans. List the strong entity types; is there a weak entity type (name, partial key, identifying relationship)?; what do its partial key and identifying relationship specify?

Worked solution

Strong entities: LOAN, BANK, ACCOUNT, CUSTOMER.

Weak entity: BANK_BRANCH — identifying relationship BRANCHES, partial key `Branch_no`.

Constraints: (1) no two branches of a bank have the same number; (2) a bank may have any number of branches, but a branch belongs to exactly one bank.

Chapter 6 — Relational Operations

Relational Algebra and the Unary Operators *SELECT*, *PROJECT* and *RENAME*

1 Relational Algebra

Relational algebra

A *procedural* language of operations, each taking one or two relations as input and producing a new relation as its result.

- Because every result is a relation, the algebra is **closed** — results can be fed back into further operations.
- A sequence of operations forms a **relational algebra expression**, whose result is the answer to a query.

Unary (one input)	Binary (two inputs)
SELECT σ	Union $\cup$, Intersection $\cap$
PROJECT Π	Set difference $-$
RENAME ρ	Cartesian product $\times$, Join $\bowtie$

Running examples. (i) the university *instructor* relation (*ID*, *name*, *dept_name*, *salary*); (ii) the COMPANY database (*EMPLOYEE*, *DEPARTMENT*, ...).

2 SELECT (σ)

SELECT

Chooses the subset of tuples of a relation that satisfies a selection condition: $\sigma_p(r)$, where p is the *selection predicate*. It is a filter — a *horizontal* partition of the relation into the tuples that satisfy the condition and those that do not.

Examples

Instructors in the Physics department:

$$\sigma_{Dept_Name='Physics'}(Instructor)$$

Employees in department 4 earning over \$25,000, or in department 5 earning over \$30,000:

$$\sigma_{(Dno=4 \wedge Salary>25000) \vee (Dno=5 \wedge Salary>30000)}(EMPLOYEE)$$

Physics instructors earning more than \$90,000:

$$\sigma_{dept_name='Physics' \wedge salary>90,000}(instructor)$$

All details of the Research department:

$$\sigma_{dept_name='Research'}(department)$$

Properties.

- The result S has the *same schema* as R .
- SELECT is commutative, so a cascade may be applied in any order: $\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$.
- $|\sigma_p(R)| \leq |R|$.
- Predicates use $=$, $\neq$, $>$, $\geq$, $<$, $\leq$, combined with $\wedge$ (and), $\vee$ (or), $\neg$ (not).

3 PROJECT (Π)

PROJECT

Returns the argument relation with certain attributes left out: $\Pi_{A_1, A_2, \dots, A_k}(r)$. It is a *vertical* partition — keep the listed columns, discard the rest — and it removes duplicate rows, since relations are sets.

Examples

$\Pi_{ID, name, salary}(instructor)$ (drops *dept_name*)

$\Pi_{Lname, Fname, Salary}(EMPLOYEE)$ and $\Pi_{Sex, Salary}(EMPLOYEE)$

Properties. PROJECT removes duplicates, so $|\Pi(R)| \leq |R|$; but if the attribute list *includes a key* of R , no two tuples can agree and $|\Pi(R)| = |R|$. Thus $\Pi_{Sex, Salary}(EMPLOYEE)$ may shrink, while $\Pi_{ID, name, salary}(instructor)$ cannot.

4 Composition of Operations

Because every result is a relation, operations compose: the argument of an operation may itself be an expression.

Names of instructors in Physics

$$\Pi_{name}(\sigma_{dept_name='Physics'}(instructor))$$

Several operations can be written either as one nested expression or as a sequence of named intermediate results. For the first name, last name and salary of employees in department 5:

$$\Pi_{Fname, Lname, Salary}(\sigma_{Dno=5}(Employee))$$

or

$$DEP5_EMPS \leftarrow \sigma_{Dno=5}(Employee)$$

$$RESULT \leftarrow \Pi_{Fname, Lname, Salary}(DEP5_EMPS)$$

5 RENAME (ρ) and Assignment ($\leftarrow$)

RENAME

$\rho_x(E)$ returns the result of expression E under the name x ; $\rho_{x(A_1, \dots, A_n)}(E)$ also renames the attributes. Useful when a query needs several operations, or when results need names to refer to.

Form	What it changes
$\rho_{S(B_1, \dots, B_n)}(R)$	both the relation name (to S) and the attribute names
$\rho_S(R)$	the relation name only
$\rho_{(B_1, \dots, B_n)}(R)$	the attribute names only

An alternate notation renames selected attributes only: $\rho_{OldName \rightarrow NewName}(r)$, e.g. $\rho_{Father \rightarrow Parent}(Paternity)$. With two or more attributes the ordering matters: $\rho_{Branch, Salary \rightarrow Location, Pay}(Employee)$. Applied to the earlier example, the 10 attributes of `DEP5_EMPS` can be renamed `F, M, L, S, B, A, SX, SAL, SU, DNO`.

Assignment ($\leftarrow$) works like assignment in a programming language: a query becomes a series of assignments to temporary relation variables, ending with the expression whose value is displayed.

6 Worked Examples

Employees(EmployeeID, Name, Department, Salary, JoiningYear)

Names and departments of employees who joined before 2020, without showing EmployeeID and Salary:

$$\Pi_{Name, Department, JoiningYear}(\sigma_{JoiningYear < 2020}(Employees))$$

Customers(CustomerID, FirstName, LastName, City, PurchaseAmount)

Renaming the City column to Location:

$$\rho_{City \rightarrow Location}(Customers)$$

Products(ProductID, ProductName, Category, Price, StockQuantity)

Name and price of all products in the Electronics category with more than 10 items in stock:

$$\Pi_{ProductName, Price}(\sigma_{Category='Electronics' \wedge StockQuantity > 10}(Products))$$

Chapter 7 — Set Operations, Binary Operations and Aggregation

Union, Intersection, Difference, Division, Cartesian Product, Join, Aggregate Functions and Grouping

1 Union Compatibility

Union compatibility

Two relations are union compatible if (1) they have the same *arity* (number of attributes), and (2) corresponding attribute domains are compatible — the i -th column of r holds the same type of values as the i -th column of s . The three set operations $\cup$, $\cap$, $-$ may only be applied to union-compatible relations.

2 UNION, INTERSECTION and SET DIFFERENCE

Union $r \cup s$ All tuples in r , in s , or in both; duplicates eliminated.
Intersection $r \cap s$ Tuples that are in *both* relations.
Difference $r - s$ Tuples in r that are *not* in s .

Courses in Fall 2017 / Spring 2018

Taught in either semester (or both) — union:

$$\Pi_{course_id}(\sigma_{sem="Fall" \wedge year=2017}(section)) \cup \Pi_{course_id}(\sigma_{sem="Spring" \wedge year=2018}(section))$$

Taught in *both* semesters — replace $\cup$ by $\cap$; taught in Fall 2017 but *not* in Spring 2018 — replace $\cup$ by $-$.

Further lecture examples: $Graduates \cap Managers$, $STUDENT \cap INSTRUCTOR$; $Graduates - Managers$, and both $STUDENT - INSTRUCTOR$ and $INSTRUCTOR - STUDENT$, which differ — difference depends on the order of operands. In the COMPANY database, the SSNs of employees who either work in department 5 (RESULT1) or supervise someone who does (RESULT2) are obtained by $RESULT1 \cup RESULT2$.

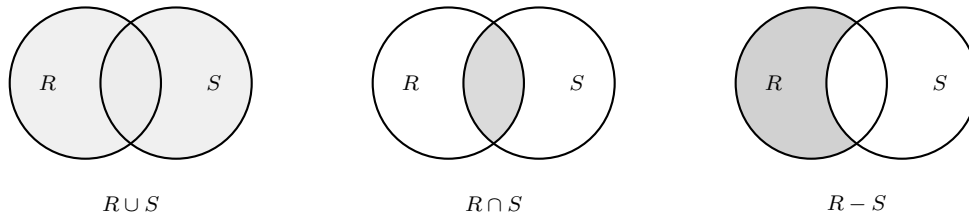


Fig 7.1 The three set operations; all require union-compatible operands.

2.1 Properties

- Union and intersection are **commutative**: $R \cup S = S \cup R$, $R \cap S = S \cap R$.
- Both are **associative**, so they extend to n -ary form: $R \cup (S \cup T) = (R \cup S) \cup T$ and $(R \cap S) \cap T = R \cap (S \cap T)$.
- Difference is **not** commutative: in general $R - S \neq S - R$.
- Intersection can be expressed with union and difference:
 $R \cap S = ((R \cup S) - (R - S)) - (S - R)$.

3 DIVISION ($\div$)

Division

A binary operation $R(Z) \div S(X)$ where $X \subseteq Z$. Let $Y = Z - X$. The result $T(Y)$ contains a tuple t if tuples t_R appear in R with $t_R[Y] = t$ and $t_R[X] = t_S$ for *every* tuple t_S in S . In words: the values in t must appear in R combined with *every* tuple of S .

SSN_PNOS		SMITH_PNOS		(b) $T \leftarrow R \div S$			
Essn	Pno	Pno		R			
123456789	1	1		A	B		
123456789	2	2		a1	b1		
666884444	3			a2	b1		
453453453	1			a3	b1		
453453453	2			a4	b1		
333445555	2			a1	b2		
333445555	3			a3	b2		
333445555	10			a2	b3		
333445555	20			a3	b3		
999887777	30			a4	b3		
999887777	10			a1	b4		
987987987	10			a2	b4		
987987987	30			a3	b4		
987654321	30						
987654321	20						
888665555	20						

SSNS (result)			
Ssn			
123456789			
453453453			

R		S		T	
A	B				
a1	b1				
a2	b1				
a3	b1				
a4	b1				
a1	b2				
a3	b2				
a2	b3				
a3	b3				
a4	b3				
a1	b4				
a2	b4				
a3	b4				

Fig 7.2 (a) $SSN_PNOS \div SMITH_PNOS$. (b) The abstract case $T \leftarrow R \div S$: only b_1 and b_4 appear in R paired with every one of a_1, a_2, a_3 .

4 Equivalent Queries

A query can be written in more than one way. Both of the following find courses taught by Physics instructors earning over 90,000, and give the same result on any database — they are **equivalent**, though not identical:

$$\begin{aligned}\sigma_{dept_name=\text{"Physics"} \wedge salary > 90,000}(instructor) \\ \sigma_{dept_name=\text{"Physics"}}(\sigma_{salary > 90,000}(instructor))\end{aligned}$$

5 Cartesian Product ($\times$)

Cartesian product

$R \times S$ combines every tuple of R with every tuple of S . If R has n attributes and S has m , the result $Q(A_1, \dots, A_n, B_1, \dots, B_m)$ has degree $n + m$; if $|R| = n_R$ and $|S| = n_S$, then $|R \times S| = n_R \times n_S$. The operands need *not* be union compatible.

For the lecture's sample tables, $EMPLOYEE \times DEPENDENT$ has 56 rows and 15 columns.

Why it is not useful alone. The product contains every combination of employees and dependents, related or not. It becomes meaningful only when a selection removes the meaningless combinations:

$$\begin{aligned}EMP_DEPENDENTS &\leftarrow EMPLOYEE \times DEPENDENT \\ ACTUAL_DEPS &\leftarrow \sigma_{Ssn=Esn}(EMP_DEPENDENTS)\end{aligned}$$

A **natural join** performs the product internally and filters automatically on columns with the same name and compatible type, so no redundant combinations survive.

6 The JOIN Operation ($\bowtie$)

Join

The join combines a SELECT and a Cartesian product into one operation. For relations $r(R)$, $s(S)$ and a predicate θ on attributes of $R \cup S$:

$$r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$$

Thus $\sigma_{instructor.id=teaches.id}(instructor \times teaches)$ is exactly $instructor \bowtie_{instructor.id=teaches.id} teaches$.

Managers and their departments

$$\begin{aligned}DEPT_MGR &\leftarrow DEPARTMENT \bowtie_{Mgr_ssn=Ssn} EMPLOYEE \\ RESULT &\leftarrow \Pi_{Dname, Lname, Fname}(DEPT_MGR)\end{aligned}$$

7 Aggregate Functions

Aggregate functions

Functions that take a collection (set or multiset) of values as input and return a single value. Basic relational algebra cannot express them, yet they are needed for statistical summaries such as average or total salary, or the number of employee tuples.

MIN()	minimum value of the resultant set
MAX()	maximum value of the resultant set
SUM()	sum of the values (numeric fields only)
AVG()	average of all non-NULL values
COUNT()	number of records; works on numeric and non-numeric data, and counts rows <i>without</i> removing duplicates

The aggregate functional operation is written $\mathcal{F}$ (or g):

$\mathcal{F}_{MAX\ Salary}(EMPLOYEE)$	maximum salary
$\mathcal{F}_{MIN\ Salary}(EMPLOYEE)$	minimum salary
$\mathcal{F}_{SUM\ Salary}(EMPLOYEE)$	sum of salaries
$\mathcal{F}_{COUNT\ SSN, AVERAGE\ Salary}(EMPLOYEE)$	count of employees and their average salary

Worked values from the lecture:

$$\begin{aligned}\mathcal{F}_{average(C)}(R) &= \frac{27}{4} = 6.75, \quad \mathcal{F}_{count(*)}(E) = 4, \\ g_{min(IC)}(R) &= 3, \quad g_{max(C)}(R) = 10.\end{aligned}$$

Aggregation over the whole relation

Total, highest, lowest and average salary, with the result attributes renamed:

$$\rho R(Total_Sal, Highest_Sal, Lowest_Sal, Average_Sal) \mathcal{F}_{SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)}(EMPLOYEE)$$

8 Grouping

Aggregates are often applied to **subgroups**: the relation is divided into non-overlapping groups (partitions) on the **grouping attribute(s)**, and the function is applied to each group independently. SQL provides `GROUP BY` for this; the grouping attributes must also appear in the `SELECT` clause so each aggregate value appears beside the group it belongs to.

Notation: grouping attributes go to the *left* of $\mathcal{F}$, aggregate functions to the *right*:

$$DNO \mathcal{F}_{COUNT SSN, AVERAGE Salary}(EMPLOYEE)$$

which groups employees by department and gives the count and average salary per department.

The three standard forms of a grouping query

Department number, number of employees and average salary, with the result attributes renamed:

$$\rho_{R(Dno, No_of_employees, Average_sal)}(Dno \mathcal{F}_{COUNT SSN, AVERAGE Salary}(EMPLOYEE))$$

Without renaming:

$$Dno \mathcal{F}_{COUNT SSN, AVERAGE Salary}(EMPLOYEE)$$

Over the whole company:

$$\mathcal{F}_{COUNT SSN, AVERAGE Salary}(EMPLOYEE)$$

Chapter 8 — Relational Algebra: Hands On

Example Queries in Procedural Form and as Single Expressions

1 COMPANY Queries

Q1 — name and address of all employees who work for the ‘Research’ department

Procedural form

$$\begin{aligned} RESEARCH_DEPT &\leftarrow \sigma_{Dname='Research'}(DEPARTMENT) \\ RESEARCH_EMPS &\leftarrow RESEARCH_DEPT \bowtie_{Dnumber=Dno} EMPLOYEE \\ RESULT &\leftarrow \Pi_{Fname, Lname, Address}(RESEARCH_EMPS) \end{aligned}$$

Single expression

$$\Pi_{Fname, Lname, Address} \left(\sigma_{Dname='Research'}(DEPARTMENT \bowtie_{Dnumber=Dno} EMPLOYEE) \right)$$

Q2 — names of employees who have no dependents

Procedural form

$$\begin{aligned} ALL_EMPS &\leftarrow \Pi_{Ssn}(EMPLOYEE) \\ EMPS_WITH_DEPS(Ssn) &\leftarrow \Pi_{Essn}(DEPENDENT) \\ EMPS_WITHOUT_DEPS &\leftarrow ALL_EMPS - EMPS_WITH_DEPS \\ RESULT &\leftarrow \Pi_{Lname, Fname}(EMPS_WITHOUT_DEPS * EMPLOYEE) \end{aligned}$$

Single expression

$$\Pi_{Lname, Fname} \left(\left(\Pi_{Ssn}(EMPLOYEE) - \rho_{Ssn}(\Pi_{Essn}(DEPENDENT)) \right) \bowtie EMPLOYEE \right)$$

Why the join at the end: after the difference only Ssn values remain, so the expression joins back to EMPLOYEE to recover the names.

2 The Sailors Database

Sailors(sid: integer, sname: string,
rating: integer, age: real)
Boats(bid: integer, bname: string, color: string)
Reserves(sid: integer, bid: integer, day: date)

Table 8.1 Underlined attributes (sid, bid, and sid, bid, day) form the key of each relation.

Sailors (instance S3)				Reserves (instance R2)			Boats (instance B1)		
<u>sid</u>	sname	rating	<u>age</u>	<u>sid</u>	<u>bid</u>	<u>day</u>	<u>bid</u>	bname	color
22	Dustin	7	45.0	22	101	10/10/98	101	Interlake	blue
29	Brutus	1	33.0	22	102	10/10/98	102	Interlake	red
31	Lubber	8	55.5	22	103	10/8/98	103	Clipper	green
32	Andy	8	25.5	22	104	10/7/98	104	Marine	red
58	Rusty	10	35.0	31	102	11/10/98			
64	Horatio	7	35.0	31	103	11/6/98			
71	Zorba	10	16.0	31	104	11/12/98			
74	Horatio	9	35.0	64	101	9/5/98			
85	Art	3	25.5	64	102	9/8/98			
95	Bob	3	63.5	74	103	9/8/98			

Table 8.2 The Sailors (S3), Reserves (R2) and Boats (B1) instances used below.

2.1 Four introductory queries (SELECT and PROJECT only)

Query	Expression
1 Rows for expert sailors (rating over 8)	$\sigma_{rating > 8}(Sailors)$
2 The rating of each sailor	$\Pi_{sname, rating}(Sailors)$
3 The age of sailors	$\Pi_{age}(Sailors)$
4 Name and rating of highly rated sailors	$\Pi_{sname, rating}(\sigma_{rating > 8}(Sailors))$

On query 3

Because age is not a key, $\Pi_{age}(Sailors)$ eliminates duplicates: 35.0 occurs three times in the instance but once in the result, and 25.5 twice but once — the duplicate-removal property of PROJECT.

2.2 Join queries

Names of sailors who reserved boat 103

$$\pi_{sname}((\sigma_{bid=103} Reserves) \bowtie Sailors)$$

Names of sailors who reserved a red boat

$$\pi_{sname}((\sigma_{color='red'} Boats) \bowtie Reserves \bowtie Sailors)$$

Colours of boats reserved by Lubber

$$\pi_{color}((\sigma_{sname='Lubber'} Sailors) \bowtie Reserves \bowtie Boats)$$

Names of sailors who reserved at least one boat

$$\pi_{sname}(Sailors \bowtie Reserves)$$

Why no selection is needed

The join does the work: a sailor appears in $Sailors \bowtie Reserves$ only if that sailor's `sid` occurs in `Reserves` — precisely “has reserved at least one boat”.

2.3 Red or green — two equivalent formulations

Union of two selections

$$\rho(Tempboats, (\sigma_{color='red'}Boats) \cup (\sigma_{color='green'}Boats))$$

$$\pi_{sname}(Tempboats \bowtie Reserves \bowtie Sailors)$$

Disjunctive selection predicate

$$\rho(Tempboats, \sigma_{color='red' \vee color='green'}Boats)$$

$$\pi_{sname}(Tempboats \bowtie Reserves \bowtie Sailors)$$

2.4 Red and green — the instructive case

INCORRECT, and why

$$\rho(Tempboats2, (\sigma_{color='red'}Boats) \cap (\sigma_{color='green'}Boats))$$

The intersection is taken over *boats*, and no single boat is both red and green, so `Tempboats2` is empty. The intersection must be taken over *sailors* — over the `sid` values — after each colour is resolved separately.

Correct formulation

$$\rho(Tempred, \pi_{sid}((\sigma_{color='red'}Boats) \bowtie Reserves))$$

$$\rho(Tempgreen, \pi_{sid}((\sigma_{color='green'}Boats) \bowtie Reserves))$$

$$\pi_{sname}((Tempred \cap Tempgreen) \bowtie Sailors)$$

2.5 At least two boats — the self-product

$$\rho(Reservations, \pi_{sid,sname,bid}(Sailors \bowtie Reserves))$$

$$\rho(Reservationpairs(1 \rightarrow sid1, 2 \rightarrow sname1, 3 \rightarrow bid1,$$

$$4 \rightarrow sid2, 5 \rightarrow sname2, 6 \rightarrow bid2),$$

$$Reservations \times Reservations)$$

$$\pi_{sname1} \sigma_{(sid1=sid2) \wedge (bid1 \neq bid2)} Reservationpairs$$

The self-product idea

To talk about *two* reservations by the same sailor, the relation is paired with itself by a Cartesian product, and renaming gives the two copies distinct attribute names. The selection then demands the same sailor ($sid1 = sid2$) but different boats ($bid1 \neq bid2$).

2.6 Negation — sailors over 20 who have *not* reserved a red boat

$$\pi_{sid}(\sigma_{age>20} Sailors) - \pi_{sid}((\sigma_{color='red'}Boats) \bowtie Reserves \bowtie Sailors)$$

The pattern for “not”

Subtract the set satisfying the positive condition from the set of all candidates. Here the candidates are sailors over 20 and the subtracted set is those who *did* reserve a red boat.

2.7 “All” — division

Sailors who have reserved all boats

$$\rho(Tempships, (\pi_{sid,bid}Reserves) \div (\pi_{bid}Boats))$$

$$\pi_{sname}(Tempships \bowtie Sailors)$$

... all boats called Interlake — same expression with the divisor restricted:

$$\rho(Tempships, (\pi_{sid,bid}Reserves) \div (\pi_{bid}(\sigma_{bname='Interlake'}Boats)))$$

The pattern for “all”

Both queries use division; only the divisor differs. Dividing by *all* boats answers “all boats”; dividing by a selected subset answers “all boats of that kind”. A `sid` survives only if it is paired in `Reserves` with every `bid` in the divisor.

3 The Pizza Database

Relation	Key
Person(name, age, gender)	name
Frequents(name, pizzeria)	(name, pizzeria)
Eats(name, pizza)	(name, pizza)
Serves(pizzeria, pizza, price)	(pizzeria, pizza)

Note from the lecture

In general there are many correct expressions for each query; the following are sample solutions.

(a) Pizzerias frequented by at least one person under 18

$$\pi_{pizzeria}(\sigma_{age < 18}(Person) \bowtie Frequent)$$

(b) Females who eat mushroom or pepperoni pizza

$$\pi_{name}(\sigma_{gender='female' \wedge (pizza='mushroom' \vee pizza='pepperoni')}(Person \bowtie Eats))$$

(c) Females who eat *both* mushroom and pepperoni pizza

$$\begin{aligned} &\pi_{name}(\sigma_{gender='female' \wedge pizza='mushroom'}(Person \bowtie Eats)) \\ &\cap \pi_{name}(\sigma_{gender='female' \wedge pizza='pepperoni'}(Person \bowtie Eats)) \end{aligned}$$

(d) Pizzerias serving a pizza Amy eats for less than \$10

$$\pi_{pizzeria}(\sigma_{name='Amy'}(Eats) \bowtie \sigma_{price < 10}(Serves))$$

(e) Pizzerias frequented by only females or only males

$$\begin{aligned} &(\pi_{pizzeria}(\sigma_{gender='female'}(Person) \bowtie Frequent) \\ &\quad - \pi_{pizzeria}(\sigma_{gender='male'}(Person) \bowtie Frequent)) \cup \\ &(\pi_{pizzeria}(\sigma_{gender='male'}(Person) \bowtie Frequent) \\ &\quad - \pi_{pizzeria}(\sigma_{gender='female'}(Person) \bowtie Frequent)) \end{aligned}$$

(f) For each person, pizzas they eat that no pizzeria they frequent serves

$$Eats - \pi_{name, pizza}(Frequent \bowtie Serves)$$

(g) People who frequent only pizzerias serving at least one pizza they eat

$$\pi_{name}(Person) - \pi_{name}(Frequent - \pi_{name, pizzeria}(Eats \bowtie Serves))$$

(h) People who frequent *every* pizzeria serving at least one pizza they eat

$$\pi_{name}(Person) - \pi_{name}(\pi_{name, pizzeria}(Eats \bowtie Serves) - Frequent)$$

(i) The pizzeria serving the cheapest pepperoni pizza (ties included)

$$\begin{aligned} &\pi_{pizzeria}(\sigma_{pizza='pepperoni'}Serves) - \\ &\pi_{pizzeria}(\sigma_{price > price2}(\pi_{pizzeria, price}(\sigma_{pizza='pepperoni'}Serves) \\ &\quad \times \rho_{pizzeria2, price2}(\pi_{pizzeria, price}(\sigma_{pizza='pepperoni'}Serves)))) \end{aligned}$$

Alternate solution using aggregation

$$\begin{aligned} A &\leftarrow \sigma_{pizza='pepperoni'}(Serves) \\ B &\leftarrow \rho_{(minprice)} \gamma_{min(price)}(A) \\ &\pi_{pizzeria}(\sigma_{price=minprice}(A \bowtie B)) \end{aligned}$$

Two routes to “cheapest”

The first finds the minimum *by elimination*: pair the pepperoni-serving pizzerias with themselves and discard every pizzeria whose price is beaten by another; ties survive because a tied price never satisfies $price > price2$. The alternate computes the minimum directly with an aggregate and selects the matching rows — which also keeps ties.

Chapter 9 — SQL Overview and Data Definition

Database Languages, Attribute Data Types and the DDL

1 Introduction to SQL

- SQL is considered one of the major reasons for the commercial success of relational databases.
- The name comes from **SEQUEL** (“SEQUEL to SQUARE”, Chamberlin and Boyce); IBM could not copyright that term, so it was abbreviated to SQL — now read as **Structured Query Language**.
- Features cover data definition, data manipulation, transaction control, indexing, security (`GRANT`, `REVOKE`), active databases, multimedia and distributed databases.

DDL	Data Definition Language — defines the database structure or schema.
DML	Data Manipulation Language — manages data within schema objects.
DCL	Data Control Language — controls access to the database.
TCL	Transaction Control Language — runs (commits) the changes made by DML statements.

2 Schema Concepts

An SQL schema is identified by a **schema name**, includes an **authorisation identifier** and descriptors for each element, and contains **schema elements**: tables, constraints, views, domains. Every SQL statement ends with a semicolon.

```
CREATE SCHEMA Library;
-- In MySQL this is the same as:
CREATE DATABASE Library;
-- PostgreSQL / Oracle:
CREATE SCHEMA Library AUTHORIZATION 'Jsmith';
```

3 Attribute Data Types

Category	Types
Numeric — integers	INTEGER, INT, SMALLINT
Numeric — floating point	FLOAT/REAL, DOUBLE PRECISION
Character string, fixed	CHAR(<i>n</i>), CHARACTER(<i>n</i>)
Character string, varying	VARCHAR(<i>n</i>), CHAR VARYING(<i>n</i>), CHARACTER VARYING(<i>n</i>)
Bit string, fixed / varying	BIT(<i>n</i>) / BIT VARYING(<i>n</i>)
Boolean	TRUE or FALSE
DATE	Ten positions: YEAR, MONTH, DAY as YYYY-MM-DD; RDBMSs provide mapping functions to change formats.
TIMESTAMP	DATE and TIME fields plus at least six positions for decimal fractions of seconds, with an optional WITH TIME ZONE qualifier.
INTERVAL	A relative value used to increment or decrement a date, time or timestamp.

DATE, TIME, TIMESTAMP and INTERVAL can be cast or converted to string formats for comparison.

3.1 The standard built-in types

char(<i>n</i>)	Fixed-length character string of length <i>n</i> (full form character).
varchar(<i>n</i>)	Variable-length string with maximum length <i>n</i> (character varying).
int	An integer — a machine-dependent finite subset of the integers (integer).
smallint	A small integer, a machine-dependent subset of int.
numeric(<i>p</i> , <i>d</i>)	Fixed-point number of <i>p</i> digits plus sign, with <i>d</i> digits after the decimal point. numeric(3,1) stores 44.5 exactly, but not 444.5 or 0.32.
real, double precision	Floating-point and double-precision floating-point with machine-dependent precision.
float(<i>n</i>)	Floating-point number with precision of at least <i>n</i> digits.
null	Every type includes the null value: a value that may exist but is unknown, or may not exist at all.
ENUM	MySQL string object limiting a column to a list of permitted values, e.g. ENUM('value_1', 'value_2').

3.2 CHAR versus VARCHAR

CHAR	VARCHAR
Use when strings are always the same size (pincode).	Use when the size varies (name).
Fixed length: storage always equals the defined length, padded with trailing blanks.	Variable length: storage equals the length of the actual data.
Slightly better retrieval performance (fixed size).	Slightly lower performance (variable sizes).
Can hold NULL if defined nullable.	Can hold NULL if defined nullable.

Storing “Avi”

In `char(10)`, seven blanks are appended to make the value ten characters long. In `varchar(10)`, no padding is added — only the three characters are stored.

3.3 Large objects — BLOB and CLOB

Applications may need attributes of many kilobytes, megabytes or gigabytes — photographs, high-resolution medical images, video clips. SQL provides large-object types for character data (clob) and binary data (blob); “LOB” = Large Object. Oracle declarations: `book_review clob(10KB)`, `image blob(10MB)`, `movie blob(2GB)`.

- Retrieving an entire large object into memory is inefficient or impractical, so LOBs are stored to optimise space and give efficient access inside the tablespaces, with transactional support (commit, rollback).
- BLOB** (Binary Long Object) holds videos, images, gifs, audio; columns are treated as binary strings using the binary character set; comparison and sorting use raw byte values; *BLOB cannot have default values*.
- CLOB** — in MySQL, `TEXT` is the equivalent; stores long textual content that will not fit in `VARCHAR`; use a proper character set (`utf8mb4`); sorting and comparison follow the character set; a prefix length must be given for indexes; *TEXT cannot have default values*.

BLOB type	Max size	TEXT type	Max size
TINYBLOB	255 bytes	TINYTEXT	255 chars
BLOB	65,535 B (~64 KB)	TEXT	65,535 chars
MEDIUMBLOB	16,777,215 B (~16 MB)	MEDIUMTEXT	16,777,215 chars
LONGBLOB	4,294,967,295 B (~4 GB)	LONGTEXT	4,294,967,295 chars

One table covering all data types

```
CREATE TABLE example (
  id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  char_column CHAR(50) NOT NULL,
  varchar_column VARCHAR(255) NOT NULL,
  tinytext_column TINYTEXT NOT NULL,
  text_column TEXT NOT NULL,
  mediumtext_column MEDIUMTEXT NOT NULL,
  longtext_column LONGTEXT NOT NULL,
  binary_column BINARY(20) NOT NULL,
  varbinary_column VARBINARY(20) NOT NULL,
  tinyblob_column TINYBLOB NOT NULL,
  blob_column BLOB NOT NULL,
  mediumblob_column MEDIUMBLOB NOT NULL,
  longblob_column LONGBLOB NOT NULL,
  enum_column ENUM('value1','value2','value3') NOT NULL,
  set_column SET('value1','value2','value3') NOT NULL,
  bool_column BOOLEAN NOT NULL DEFAULT 0,
  date_column DATE NOT NULL,
  datetime_column DATETIME NOT NULL,
  timestamp_column TIMESTAMP NOT NULL
    DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  time_column TIME NOT NULL,
  year_column YEAR(4) NOT NULL,
  float_column FLOAT(10,6) NOT NULL DEFAULT '0.000000',
  double_column DOUBLE(10,6) NOT NULL DEFAULT '0.000000'
);
```

4 SQL Data Definition (DDL)

Terminology: **table**, **row** and **column** are the SQL terms for relation, tuple and attribute. `CREATE` is the main data-definition command; `CREATE TABLE` creates a new relation by naming it and specifying its attributes and initial constraints.

<code>CREATE</code>	create objects in the database
<code>ALTER</code>	alter the structure of the database
<code>DROP</code>	delete objects from the database
<code>TRUNCATE</code>	remove all records from a table, including the space allocated for them
<code>COMMENT</code>	add comments to the data dictionary
<code>RENAME</code>	rename an object

```
CREATE TABLE table_name(
  Col_name1 datatype(),
  Col_name2 datatype(),
  ... Col_namen datatype() );
```

```
-- from an existing table
CREATE TABLE new_table AS SELECT * FROM existing_table;
CREATE TABLE new_table LIKE existing_table;
```

The distinction the lecture draws

What is the difference between `CREATE TABLE t AS SELECT * FROM e;` and `CREATE TABLE t LIKE e;`? (The first copies structure *and* rows; the second copies only the structure.)

4.1 ALTER, DESCRIBE, DROP, RENAME, TRUNCATE

```
-- 1) ADD
ALTER TABLE table_name ADD Col_name datatype()...;
-- 2) MODIFY
ALTER TABLE table_name MODIFY (fieldname datatype()...);
```

```
-- 3) RENAME
ALTER TABLE table_name
    RENAME COLUMN (Old_fieldname TO New_fieldname...);
-- 4) DROP
ALTER TABLE table_name DROP column_name;

-- DESCRIBE
DESCRIBE table_name;          -- basic description
SHOW CREATE TABLE table_name; -- full information,
                                -- constraints, indexes

-- DROP / RENAME / TRUNCATE
DROP TABLE table_name;      -- structure is dropped too
RENAME TABLE table_name TO new_table_name;
TRUNCATE TABLE table_name; -- data removed,
                                -- structure remains
```

Feature	DROP	TRUNCATE
Removes data	Yes	Yes
Removes structure	Yes (table deleted)	No (structure remains)
AUTO_INCREMENT	N/A (table gone)	Resets to 1 (MySQL)
Constraints & indexes	Deleted with the table	Preserved (only rows deleted)
Rollback (MySQL)	Cannot be rolled back	Cannot be rolled back
Use case	Table no longer needed	Empty the table but keep its structure

5 Defining the COMPANY Schema

```
CREATE TABLE `department` (
    `Dname` varchar(15) NOT NULL,
    `Dnumber` int(11) NOT NULL,
    `Mgr_ssn` char(9) NOT NULL,
    `Mgr_start_date` date DEFAULT NULL);

CREATE TABLE `dependent` (
    `Essn` char(9) NOT NULL,
    `Dependent_name` varchar(15) NOT NULL,
    `Gender` char(1) DEFAULT NULL,
    `Bdate` date DEFAULT NULL,
    `Relationship` varchar(8) DEFAULT NULL);

CREATE TABLE `dept_locations` (
    `Dnumber` int(11) NOT NULL,
    `Dlocation` varchar(15) NOT NULL);

CREATE TABLE `employee` (
    `Fname` varchar(15) NOT NULL,
    `Minit` char(1) DEFAULT NULL,
    `Lname` varchar(15) NOT NULL,
    `Ssn` char(9) NOT NULL,
    `Bdate` date DEFAULT NULL,
    `Address` varchar(30) DEFAULT NULL,
    `Gender` char(1) DEFAULT NULL,
    `Salary` decimal(10,2) DEFAULT NULL,
    `Super_ssn` char(9) DEFAULT NULL,
    `Dno` int(11) NOT NULL);

CREATE TABLE `project` (
    `Pname` varchar(15) NOT NULL,
    `Pnumber` int(11) NOT NULL,
    `Plocation` varchar(15) DEFAULT NULL,
    `Dnum` int(11) NOT NULL);

CREATE TABLE `works_on` (
    `Essn` char(9) NOT NULL,
    `Pno` int(11) NOT NULL,
    `Hours` decimal(3,1) DEFAULT NULL);
```

6 Comments in DDL

Comments can be attached to a table or a column; they serve as metadata describing database objects, and `SHOW CREATE TABLE table_name` displays them.

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(255)
) COMMENT 'This table stores information about users.';

CREATE TABLE products (
    product_id INT PRIMARY KEY
    COMMENT 'Unique identifier for the product',
    product_name VARCHAR(255) COMMENT 'Name of the product',
    price DECIMAL(10,2) COMMENT 'Price of the product' );
```

Chapter 10 — Constraints and Schema Change Statements

Constraints in SQL, Referential Actions, ALTER, DROP, TRUNCATE and RENAME

1 Constraints in SQL

Constraints

Rules enforced on data columns of a table, used to prevent invalid data from entering the database and so to ensure the accuracy and reliability of the data.

Key constraints	No two tuples may agree on the key attributes.
Entity integrity	Primary-key attributes may not be NULL.
Referential integrity	A foreign-key value must match an existing primary-key value, or be NULL.

1.1 Attribute constraints and defaults

- **NOT NULL** — the column may not store NULL; specified when NULL is not permitted for that attribute.
- **DEFAULT <value>** — appended to an attribute definition, gives the value used when none is supplied.
- **CHECK** — restricts the values in a column, e.g.

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
```

1.2 Naming constraints

The keyword **CONSTRAINT** names a constraint; a name is needed if the constraint is later to be dropped or replaced.

```
CREATE TABLE EMPLOYEE (
    ...,
    CONSTRAINT EMPPK PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK FOREIGN KEY (Super_ssn)
        REFERENCES EMPLOYEE(Ssn)
        ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK FOREIGN KEY (Dno)
        REFERENCES DEPARTMENT(Dnumber)
        ON DELETE SET DEFAULT ON UPDATE CASCADE );
```

```
CREATE TABLE DEPT_LOCATIONS (
    Dnumber INT NOT NULL DEFAULT 1,
    Dlocation VARCHAR(15) NOT NULL DEFAULT 'Bangalore',
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
        ON DELETE CASCADE ON UPDATE CASCADE );
```

2 Referential Triggered Actions

```
FOREIGN KEY (column_name)
    REFERENCES parent_table(parent_column)
    ON DELETE <action>
    ON UPDATE <action>;
```

- The designer may attach a referential triggered action to any foreign-key constraint; the options are **SET NULL**, **CASCADE** and **SET DEFAULT**, each qualified by **ON DELETE** or **ON UPDATE**.
- The **default action**, if no clause is given, is to *reject* the operation that would violate referential integrity.
- **CASCADE** suits “relationship” relations such as **WORKS_ON**.

Option (ON DELETE)	Behaviour when the referenced row is deleted
NO ACTION	An error is raised and the DELETE is rolled back.
CASCADE	All rows containing those foreign keys are also deleted.
SET NULL	All rows containing those foreign keys are set to NULL.
SET DEFAULT	Those foreign keys are set to their default value.

Option (ON UPDATE)	Behaviour when the referenced key is updated
NO ACTION	An error is raised and the UPDATE is rolled back.
CASCADE	All foreign-key values are updated to the new key value.
SET NULL	All foreign keys referencing that key are set to NULL.
SET DEFAULT	All foreign keys referencing that key are set to their default.

2.1 CHECK constraints on tuples

A **CHECK** clause at the end of a **CREATE TABLE** statement constrains individual tuples — e.g. **CHECK (Dept_create_date <= Mgr_start_date)**; the department’s creation date must not be later than the manager’s start date.

3 Schema Change Statements

Schema-evolution commands are available in most systems, can be run *while the database is operational*, and need no recompilation of the schema.

CASCADE	Removes the schema and all of its elements — tables, views, constraints.
RESTRICT	The schema is removed only if it has no elements; otherwise DROP is not executed.

```
DROP SCHEMA COMPANY CASCADE; -- schema + all elements
DESCRIBE table_name; SHOW COLUMNS FROM table_name;
DESC table_name;
```

```
USE database_name;
DROP DATABASE database_name;
DROP DATABASE IF EXISTS database_name;
```

4 The ALTER TABLE Command

ALTER TABLE adds, deletes or modifies columns of an existing table, and adds or drops constraints.

```
-- Form 1: add a single column
ALTER TABLE table_name ADD col_name datatype;
-- Form 2: add multiple columns
ALTER TABLE table_name
  ADD col_name1 datatype, ADD col_name2 datatype;
-- Form 3: modify a column definition
ALTER TABLE table_name MODIFY col_name new_datatype;
ALTER TABLE t MODIFY col CHAR(10);
ALTER TABLE t MODIFY j BIGINT NOT NULL DEFAULT 100;
-- Form 5: rename a column (CHANGE needs the
--          full definition)
ALTER TABLE table_name
  CHANGE COLUMN old_col_name new_col_name datatype;
-- position
ALTER TABLE table_name
  MODIFY col_name datatype AFTER other_col;
ALTER TABLE table_name MODIFY col_name datatype FIRST;
-- Form 4: drop a column
ALTER TABLE table_name DROP COLUMN col_name;
ALTER TABLE EMPLOYEE DROP COLUMN Address CASCADE;
-- defaults
ALTER TABLE t ALTER i SET DEFAULT 1000;
ALTER TABLE DEPARTMENT ALTER COLUMN Mgr_ssn DROP DEFAULT;
ALTER TABLE DEPARTMENT ALTER COLUMN Mgr_ssn
  SET DEFAULT '333445555';
-- tables
DROP TABLE table_name; DROP TABLE IF EXISTS table_name;
DROP TABLE table1, table2, table3;
TRUNCATE TABLE table_name;
RENAME TABLE old_table_name TO new_table_name;
RENAME TABLE products TO products_old,
  products_new TO products;
```

Two points from the lecture

Existing rows: ALTER TABLE R ADD A D; assigns NULL to the new attribute in every existing tuple.

Limitation: alter table r drop A; — dropping attributes is not supported by many databases. Dropping a column may also carry CASCADE (constraints and views referencing the column are dropped with it) or RESTRICT (succeeds only if nothing references the column).

To drop a constraint it must have been *named* when specified.

5 ALTER — A Worked Sequence

```
-- 1) add a column
ALTER TABLE employee ADD email VARCHAR(100);
-- 2) reposition it
ALTER TABLE employee MODIFY email VARCHAR(100) AFTER name;
ALTER TABLE employee MODIFY email VARCHAR(100) FIRST;
-- 3) rename it
ALTER TABLE employee
  CHANGE COLUMN email_email_id VARCHAR(100);
-- 4) change a data type
ALTER TABLE employee MODIFY dept_id SMALLINT;
-- 5a) NOT NULL
ALTER TABLE employee MODIFY name VARCHAR(50) NOT NULL;
-- 5b) DEFAULT
ALTER TABLE employee ALTER dept_id SET DEFAULT 100;
-- 5c) named CHECK
ALTER TABLE employee
  ADD CONSTRAINT chk_dept CHECK (dept_id > 0);
-- 5d) primary key swap
ALTER TABLE employee DROP PRIMARY KEY;
ALTER TABLE employee ADD PRIMARY KEY (emp_id);
-- 5e) foreign keys with different referential actions
ALTER TABLE employee ADD CONSTRAINT fk_dept
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
  ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE employee ADD CONSTRAINT fk_null
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
  ON DELETE SET NULL;
ALTER TABLE employee ADD CONSTRAINT fk_default
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
  ON DELETE SET DEFAULT;
```

```
-- 6) dropping them again
ALTER TABLE employee ALTER dept_id DROP DEFAULT;
ALTER TABLE employee DROP CHECK chk_dept;
ALTER TABLE employee DROP PRIMARY KEY;
ALTER TABLE employee DROP FOREIGN KEY fk_dept;
ALTER TABLE employee DROP COLUMN email_id;
```

Chapter 11 — Database Modification

INSERT, SELECT, UPDATE and DELETE

1 Data Manipulation Language

DML

Statements that manage data within schema objects — **SELECT** (retrieve), **INSERT** (add), **UPDATE** (modify) and **DELETE** (remove) — used to store, modify, retrieve, delete and update data.

2 INSERT

In its simplest form **INSERT** adds a single tuple to a relation.

Form 1 — positional	Form 2 — explicit attribute list
No attribute list.	Attributes named before VALUES .
Values must follow the exact order of the CREATE TABLE definition.	Columns that allow NULL or have a DEFAULT may be omitted.
A value is needed for every attribute.	Only NOT NULL columns without a default must appear.

```
-- Form 1
INSERT INTO Table_name
VALUES ('value1','value2', ... , 'valuen');
INSERT INTO EMPLOYEE
VALUES ('Richard','K','Marini','653298653','30-DEC-52',
      '98 Oak Forest, Katy, TX','M',37000,
      '653298653',4);

-- Form 2
INSERT INTO Table_name (col_name1, ... , col_namen)
VALUES ('value1', ... , 'valuen');
INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard','Marini',4,'653298653');
```

2.1 INSERT with a SELECT

A variation inserts many tuples at once, loading a relation with the result of a query.

```
CREATE TABLE WORKS_ON_INFO (
  Emp_name VARCHAR(15),
  Proj_name VARCHAR(15),
  Hours_per_week DECIMAL(3,1) );

INSERT INTO WORKS_ON_INFO
  (Emp_name, Proj_name, Hours_per_week)
SELECT E.Lname, P.Pname, W.Hours
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;

-- taking a backup of a table
INSERT INTO WORKS_ON_INFO_Backup
  (SELECT * FROM WORKS_ON_INFO);
```

The backup table must already exist and have the same schema as the source table.

3 SELECT

SELECT fetches data from tables and returns it as a result table.

SELECT	Corresponds to the <i>projection</i> of relational algebra: lists the attributes wanted in the result.
FROM	Corresponds to the <i>Cartesian product</i> : lists the relations to be scanned.
WHERE	Corresponds to the <i>selection predicate</i> : a condition over attributes of the relations in FROM .

```
SELECT col_name1, col_name2, ...
FROM table_name
WHERE <condition>;
```

```
SELECT fname, Lname FROM EMPLOYEE;
```

4 UPDATE

UPDATE modifies attribute values of selected tuples: a **WHERE** clause chooses the tuples of one relation, and **SET** names the attributes and their new values. Several tuples may be modified by one command, and updating a primary key may propagate to foreign keys elsewhere if a referential triggered action says so.

```
UPDATE table_name
SET col_name1 = value1, col_name2 = value2, ...
WHERE <condition>;      -- omit WHERE to update every row

-- 1) change location and controlling department
--    of project 10
UPDATE PROJECT SET PLOCATION = 'Bellaire', DNUM = 5
WHERE PNUMBER = 10;

-- 2) 10% raise for the 'Research' department
UPDATE EMPLOYEE SET SALARY = SALARY * 1.1 WHERE DNO = 5;
```

Reading example 2

The new `SALARY` depends on the old one, so `SALARY` appears twice: on the right of the assignment it means the old value, on the left the new value.

Example 3 — 5% raise for employees earning below the average

MySQL does not allow a subquery in `WHERE` to read the table being updated, so the subquery is wrapped in a derived table:

```
UPDATE Employee SET salary = salary * 1.05
WHERE salary < (
  SELECT avg_salary FROM (
    SELECT AVG(salary) AS avg_salary FROM Employee
  ) AS temp );

-- PostgreSQL / Oracle / SQL Server take
-- the direct form
UPDATE Employee SET salary = salary * 1.05
WHERE salary < (SELECT AVG(salary) FROM Employee);
```

5 DELETE

- Removes tuples from *one* relation at a time, selected by a `WHERE` clause; zero, one or many tuples may be deleted.
- Deletion may propagate to other relations if referential triggered actions are specified.
- A *missing* `WHERE` clause deletes all tuples but leaves the table in the database as an empty table; `DROP TABLE` is needed to remove the table definition itself.

```
DELETE FROM table_name WHERE <condition>;
DELETE FROM EMPLOYEE WHERE LNAME = 'Zelaya';
DELETE FROM EMPLOYEE;    -- empties the table, keeps it
DROP TABLE EMPLOYEE;    -- removes rows and definition
```